Fidelity Synopses: Pre-Execution Plan Selection for Approximate Vector Search

Adithya Ramanatha
NIT Calicut
Calicut, India
adithyar2115@gmail.com

Sanskar S. Singh
IIT Delhi
New Delhi, India
ssanskar2705@gmail.com

Maya Ramanath
IIT Delhi
New Delhi, India
ramanath@cse.iitd.ac.in

ABSTRACT

Vector search has become a database operator with many physical implementations: different index families, build-time and search-time parameters, and compressed or disk-resident variants. Each trades cost against *fidelity*, the fraction of a query's exact top-k nearest neighbors that a configuration returns. Deployments nevertheless choose one configuration offline and use it for every query, so they are sized for their hardest queries and overpay for the rest. On a 5.8M-item LAION benchmark with four embedding models, running each query on the cheapest configuration that suffices for it would roughly halve the cost in a typical setting. Exploiting this requires predicting each configuration's fidelity before running it, which is hard because fidelity is defined by the very exact search that approximate indexes avoid. Relational optimizers solve the analogous problem for cardinality with offline synopses; we propose one for vector search. A *fidelity synopsis* divides the embedding space into cells around a set of landmarks and records, for every cell and configuration, how often that configuration met a fidelity target on a sample of queries. At query time, one nearest-landmark lookup yields a calibrated estimate for every configuration, and an optimizer chooses configurations so that the workload misses the target no more often than its requirement allows. We test the two premises the synopsis relies on, that fidelity varies smoothly across the embedding space and that its loss is specific to each configuration. Held to the same requirement a static deployment is built to meet, synopsis-guided selection lowers cost in all 16 combinations of model and query direction, to a median of 0.72× the static deployment, with a batched lookup of 21–32 μs per query. In one of them, no single configuration meets the requirement, but the synopsis does. Costs are estimated from per-configuration latencies measured with full batches. The saving persists against the best static choice among the same indexes, and within a single HNSW index searched at several settings, a practical deployment that needs no extra memory, wherever that index can meet the requirement.

	Queries whose cheapest sufficient plan costs					Optimum
Model (text→image)	≤ 0.33×	0.33–1×	1×	> 1×	none	cost
Jina-v4	**58.3%**	25.7%	10.1%	1.2%	4.7%	0.41×
Qwen3-VL	**76.8%**	0.0%	18.2%	3.6%	1.4%	0.39×
SigLIP2	**89.7%**	0.0%	7.7%	1.6%	1.0%	0.21×
OpenCLIP	**88.8%**	0.0%	9.6%	1.5%	0.1%	0.20×

Table 1: Text-to-image search at a fidelity target of 0.7 on 5.8M LAION items, with all costs relative to the static plan. Left: held-out queries by the cost of their cheapest sufficient configuration (bold: at most a third of the static plan; none: no configuration meets the target). Right: the mean cost if every query ran on its cheapest sufficient configuration.

1 INTRODUCTION

Dense vector representations have become the common currency of search. Text retrievers such as DPR [19] and ColBERT [20] map queries and documents into a shared embedding space, and contrastive vision–language models such as CLIP [37] and SigLIP [49], together with more recent multimodal embedders [26, 47], do the same across images, text, and visually rich documents, so that a text query can retrieve images and vice versa. In all of these settings, answering a query means finding its nearest neighbors in a large collection of vectors. Exact search compares the query with every vector and quickly becomes too slow; approximate nearest-neighbor (ANN) indexes such as HNSW [30], IVFPQ [15], and O-IVFPQ [9] return most of the true neighbors at a fraction of the cost. The same indexes underpin retrieval-augmented generation [24].

Vector search has consequently moved into database systems. Purpose-built vector databases such as Milvus [43], analytical engines such as AnalyticDB-V [46], and extensions such as pgvector [41] for PostgreSQL treat similarity search as a query operator; VBase [51] and ACORN [36] study how it composes with relational predicates, and [34] surveys the landscape. These systems offer many physical implementations of the same operator: a collection can be indexed with different index families, each with build-time parameters (graph degree, number of partitions, code length) and search-time parameters (`efSearch` for HNSW, `nprobe` for IVF indexes), kept in memory or compressed and placed on disk [7, 22]. Each choice trades latency, memory, and energy against *fidelity*, the fraction of a query's exact top-k nearest neighbors that the chosen *configuration* (an index family with its parameter values) actually returns.

Semantic query processing systems such as LOTUS [35] and Palimpzest [28] go further and compose retrieval with LLM-based filters, maps, and joins in declarative pipelines that are optimized for cost and quality; related systems process multimodal data under accuracy budgets [16] or optimize document-processing pipelines [40]. In these systems and in vector databases alike, however, the vector-search operator itself is executed in one fixed way. A configuration is chosen when the collection is indexed, typically by tuning for average recall on a sample workload, and every query then runs under it. The search-time parameters are exposed, but as session- or collection-level settings; no component chooses them per query. In relational terms, the operator has several access paths but no optimizer that chooses among them.

A fixed configuration is costly because queries differ in how hard they are to approximate. To measure this, we indexed 5.8M

LAION image–caption pairs [38] under four embedding models and nine configurations of three index families, and recorded, for every held-out query, which configurations reach a given fidelity. Table 1 shows text-to-image search at a fidelity target of 0.7. A *static plan*, the cheapest configuration that meets the target on at least 90% of queries, must be provisioned for the hardest of those queries. Yet for every model, most queries would meet the target on a configuration costing at most a third as much, and running each query on the cheapest configuration that suffices for it would cost less than half as much as the static plan.

Early-termination methods address part of this problem. LAET [25] learns to predict, per query, when a search can stop, and DARTH [4] terminates each search once a declared target recall is predicted to be met. Ada-ef [50] and Quake [31] similarly set a search parameter per query toward a recall target. All of these adapt how far a search goes within an index that has already been chosen, and none of them can help when the chosen index cannot reach the target at all, and DARTH explicitly assumes that it can. Even within a single index, they tune each search while it runs, toward a target for that query. A synopsis decides before any search starts and holds the whole workload to a miss rate. It can therefore send a query to a cheap setting wherever its cell makes a miss unlikely, and spend the workload's allowance where it is safe. In our measurements this assumption often fails. Even the most accurate HNSW configuration in our catalog reaches a fidelity of 0.7 on only 69% of OpenCLIP image-to-text queries, and for some text-to-image queries no configuration reaches it at all (Table 1, column "none"), consistent with the observation that cross-modal queries fall outside the distribution an index is built over [6, 27]. Measures of query hardness, such as local intrinsic dimensionality [3] and Steiner-hardness [45], describe how difficult a query is, but they are not calibrated estimates of fidelity under a specific configuration and cannot tell an optimizer which configuration will suffice. What is missing is a way to predict, before running anything, how each candidate configuration will perform on a given query. This is hard for a basic reason: fidelity is defined relative to the exact top-k, which is precisely what approximate indexes exist to avoid computing.

Our approach. Relational optimizers face the same problem when choosing among access paths: they estimate each plan's output size from synopses, such as histograms, that are built offline and consulted at plan time. We propose such a synopsis for vector search, which we call a *fidelity synopsis* (Figure 1). It divides the embedding space into cells around a set of *landmarks*, representative points obtained by clustering the corpus. Offline, a sample of queries is run once through exact search and through every configuration in the catalog, and for every pair of cell and configuration the synopsis records the distribution of fidelity among the sample queries in that cell. At query time, the query's nearest landmark identifies its cell, and the synopsis gives, for every configuration at once, the estimated probability that it meets the fidelity target, before any configuration is run. An optimizer then runs each query on a cheap configuration where that probability is high and on a more reliable one where it is low, so that the workload as a whole misses the target no more often than its requirement allows. Exact search is paid once, offline, on the sample.

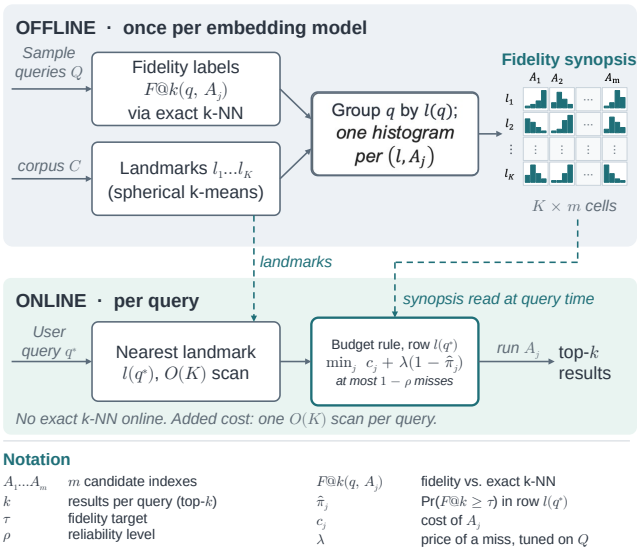


Figure 1: Overview of the fidelity synopsis. Offline (top), a sample of queries is labeled with the fidelity of every configuration, and the labels are summarized per cell and configuration. Online (bottom), a query is located in its cell, and the budget rule uses that cell's statistics to choose the configuration that runs it.

The synopsis rests on two premises, which we state and test rather than assume. The first is *smoothness*: fidelity varies slowly enough across the embedding space that a cell's statistics represent the queries that fall into it. The second is *configuration specificity*: how the local geometry of the data translates into lost fidelity differs between index families, since graph navigation and quantization fail in different ways, so the synopsis must keep separate statistics for every configuration.

Contributions. This paper makes the following contributions.

- We identify per-query choice of ANN configuration as an optimization opportunity and quantify it across four embedding models, nine configurations, and four combinations of query and corpus modality: serving the same queries as a static plan, the per-query optimum costs a median of about half as much (Section 5.1).
- We cast pre-execution fidelity estimation as the vector-search analogue of cardinality estimation and propose the fidelity synopsis, which stores $O(K(d + mB))$ numbers for K landmarks in d dimensions, m configurations, and B histogram bins, together with a selection rule that holds the workload's violation rate to a requirement (Section 3).
- We test the synopsis's two premises directly. A query's cell predicts its fidelity in every setting, the synopsis's estimate of how often a configuration meets the target is well calibrated, and keeping statistics per configuration beats a single difficulty value per cell for every model and index family (Sections 5.2 and 5.3).
- We show that, under the same requirement a static plan is built to meet, synopsis-guided selection is cheaper in all 16

settings, at a median of 0.72× the static plan (in one, the static plan itself falls short of the requirement), and that it is still cheaper in 12 of 14 feasible settings when held to the static plan's own violation rate, where no allowance is left to spend; that the saving persists against the best static choice among the same indexes, and within a single index at no extra memory, the practical deployment of Section 6, in the 13 settings where that index can meet the requirement; and where its limits lie: variation within cells caps the saving, and a single-query lookup by exact landmark scan is too slow for 2,048-dimensional models (Sections 5.4 and 5.5).

Section 2 defines fidelity and the plan-selection problem, Section 3 presents the fidelity synopsis, Section 4 describes the evaluation setup, Section 5 answers five research questions, Section 6 describes a realistic deployment, Section 7 discusses related work, and Section 8 concludes.

2 PRELIMINARIES

This section fixes the setting and notation, defines fidelity, and states the plan-selection problem. Table 2 summarizes the notation.

2.1 Corpus, Models, and Queries

A corpus C is a collection of items, each an image, a text, or both. An embedding model M maps items and queries of either modality into a shared vector space, in which similarity is measured by a fixed function; in our experiments, embeddings are L2-normalized and compared by Euclidean distance, which ranks items exactly as cosine similarity does. Each model induces its own embedding space and hence its own nearest neighbors, so everything below is defined for a single model, which we leave implicit.

A query q is an image or a text, embedded by the same model as the corpus. Searching a corpus of one modality with a query of one modality gives four *query directions*: image-to-image (i→i), text-to-text (t→t), image-to-text (i→t), and text-to-image (t→i). The first two are *uni-modal*, the last two *cross-modal*. The distinction matters because an approximate index is built over the distribution of the corpus vectors. Uni-modal queries come from roughly that distribution; cross-modal queries often do not, because contrastively trained models place the two modalities in partly separated regions of the embedding space [27], so a query of one modality may lie far from the items of the other [6]. The extent of this separation varies across models. We call a pair of model and query direction a *setting*.

2.2 Index Families and Configurations

Approximate nearest-neighbor (ANN) indexes avoid comparing a query with every item, at the price of sometimes missing true neighbors. We consider three widely deployed index families, which approximate search in different ways and therefore lose accuracy for different reasons.

HNSW. HNSW [30] builds a multi-layer proximity graph over the corpus vectors and answers a query by greedy best-first traversal, keeping a candidate list of size `efSearch`. Its build-time parameters are the graph degree M and `efConstruction`. HNSW keeps full-precision vectors, so its losses come from navigation: the traversal can stop in a region that does not contain all true neighbors, particularly for queries far from the data the graph was built on [6, 45].

IVFPQ. IVFPQ [15] partitions the corpus around n_{list} centroids and compresses each vector with product quantization into a short code. A query probes the `nprobe` partitions whose centroids are closest to it and ranks their contents by approximate distance. It loses accuracy in two ways: true neighbors may lie in partitions that are not probed, and quantization may misorder those that are.

O-IVFPQ. O-IVFPQ applies a learned rotation (OPQ [9]) before product quantization, which reduces quantization error for a given code length; otherwise it behaves like IVFPQ.

Configurations, catalog, and cost. A *configuration* A is an index family together with values for its build-time and search-time parameters. A *catalog* $\mathcal{A} = \{A_1, \ldots, A_m\}$ is the set of configurations available for a corpus and model. Configurations that differ only in search-time parameters share one built index; all others need separate indexes. Each configuration has a per-query cost $c(A)$, such as its mean search latency on a given platform. Cost depends on the model, for instance through the embedding dimension, so the cost order of a catalog can differ between models. Memory is a per-index cost and is reported separately.

2.3 Fidelity

Let Top-$k_{\text{KNN}}(q)$ be the k items of C nearest to q, computed by exact search, and Top-$k_A(q)$ the k items returned by configuration A. The *fidelity* of A on q is

$$\mathrm{F}(q, A) \;=\; \frac{\left|\,\text{Top-}k_{\text{KNN}}(q) \;\cap\; \text{Top-}k_A(q)\,\right|}{k} \;\in\; [0, 1].$$

Fidelity is what the ANN literature calls recall@k against exact search; we use a separate name to distinguish it from recall against human relevance judgments. It isolates the loss caused by approximation: the model determines which items are nearest, and fidelity measures how many of them a configuration recovers. Computing it requires the exact top-k, which is affordable offline for a sample of queries but is exactly what approximate indexes avoid at query time.

A configuration is *dominated* if another configuration is no more expensive and has at least its mean fidelity, with one of the two strictly better, for the same model and query direction. A dominated configuration is never the better choice for the workload as a whole, but it can still be the cheapest configuration that meets a target on individual queries.

2.4 Problem: Per-Query Plan Selection

An operator fixes a *fidelity target* $\tau \in (0, 1]$ and a *reliability level* $\rho \in (0, 1]$: a query should receive at least a fraction τ of its exact top-k, and at most a fraction $1 - \rho$ of the workload may fall short. We call the latter the *requirement*. For a query q, the *sufficient* configurations are those that meet the target,

$$S_\tau(q) \;=\; \{\, A \in \mathcal{A} \;:\; \mathrm{F}(q, A) \geq \tau \,\}.$$

Symbol	Meaning
C, M	corpus; embedding model
$\mathcal{A} = \{A_1, \ldots, A_m\}$	catalog of configurations
$c(A)$	per-query cost of configuration A
k	result depth
$\mathrm{F}(q, A)$	fidelity of A on q
τ, ρ	fidelity target; reliability level
$S_\tau(q)$	configurations sufficient for q
W	workload
$A_{\text{static}}, A^*(q)$	static plan; per-query optimum
$\hat{A}(q)$	selected configuration
c_{est}	per-query cost of selecting it
L, K	landmarks; their number
$\ell(q)$	q's nearest landmark (its cell)
Q, Q_{test}	construction sample; held-out workload
$\mu_{ij}, \pi_{ij}(\tau)$	mean fidelity and coverage of A_j in cell i
$\hat{F}, \hat{\pi}$	estimated fidelity; estimated coverage
λ	price of an expected miss (budget rule)

Table 2: Notation.

Static plan. A static deployment runs every query of a workload W on one configuration. The *static plan* is the cheapest configuration that meets the requirement on W,

$$A_{\text{static}} \;=\; \arg\min_{A \in \mathcal{A}} \; c(A) \quad \text{s.t.} \quad \frac{|\{q \in W : A \in S_\tau(q)\}|}{|W|} \;\geq\; \rho,$$

or, if no configuration meets it, the one that meets the target on the largest fraction of W. Because a catalog offers only a few configurations, the static plan usually exceeds the requirement: it meets the target on more than a fraction ρ of queries, because the next cheaper configuration meets it on fewer.

Per-query optimum. If every query's fidelity under every configuration were known in advance, each query could run on its cheapest sufficient configuration,

$$A^*(q) \;=\; \arg\min_{A \in S_\tau(q)} \; c(A).$$

This *per-query optimum* is not deployable, since it requires every query's exact top-k, but its cost bounds what any per-query choice can save. Section 4.5 defines the variants we compare against.

Selection. A deployable *selector* chooses a configuration $\hat{A}(q)$ for each query from estimates that are available before any configuration runs and that cost c_{est} per query to obtain. A *violation* occurs when $\hat{A}(q) \notin S_\tau(q)$, that is, when the chosen configuration misses the target. The goal is the selector with the lowest mean cost $c_{\text{est}} + c(\hat{A}(q))$ whose violation rate meets the requirement, at most $1 - \rho$: the same requirement the static plan is built to meet. As a more conservative comparison, we also hold the selector to the violation rate the static plan actually incurs, which is usually lower. The two goals trade off: a selector that always picks the cheapest configuration minimizes cost but violates often, while the static plan meets the requirement at the cost of serving every query as if it were among the hardest.

3 THE FIDELITY SYNOPSIS

3.1 Overview and Intuition

The fidelity of a configuration on a query is determined by the geometry of the corpus around the query: how densely its true neighbors are packed, how far they lie from it, whether they fall within one IVF partition or straddle several, and how well connected the part of the HNSW graph that contains them is. Two queries that land close to each other in the embedding space largely share these surroundings, and often many of their true neighbors, so we expect them to fare similarly under a given configuration.

This suggests estimating fidelity by location. We divide the embedding space into regions and record, for each region and each configuration, how that configuration performed on a sample of queries that fell into the region; a new query is located and the record of its region is read off. This is the idea of a histogram in a relational optimizer, which records statistics per bucket of a column's domain so that the selectivity of a new predicate can be read off without scanning the table. Here the buckets are regions of the embedding space, and the statistic is fidelity.

Two properties make this more than a density estimate. First, the statistics are measured, not modeled: they capture whatever combination of density, partitioning, and graph structure determines fidelity in a region, without our having to know which. Second, they are kept separately for every configuration, because the same region can be easy for one index family and hard for another: a dense region whose neighbors straddle partition boundaries can be hard for IVFPQ and easy for HNSW, and a sparse region far from most data can be the reverse.

The approach rests on two premises, which Sections 5.2 and 5.3 test directly:

- *Smoothness*: fidelity varies slowly enough across the space that the sample queries in a region represent the queries that will fall into it later. This is least plausible near the boundary between regions and for queries far from all data, such as cross-modal queries under models that separate the modalities strongly.
- *Configuration specificity*: the same region can have very different fidelity under different configurations, so statistics must be kept per pair of region and configuration.

The synopsis is built once per model in an offline stage and consulted once per query in an online stage (Figure 1). Only the offline stage uses exact search, and only on a sample of queries.

3.2 Offline Stage: Building the Synopsis

The offline stage takes a model's corpus embeddings, its catalog $\mathcal{A} = \{A_1, \ldots, A_m\}$, and a *construction sample* Q (a set of queries for which exact search is affordable), and builds the synopsis in four steps. Figure 2 shows the result on real data.

Step 1: Landmarks. We choose $K \ll |C|$ *landmarks* $L = \{\ell_1, \ldots, \ell_K\}$, points in the embedding space obtained by spherical k-means on the corpus vectors. Each landmark defines a *cell*, the set of points closer to it than to any other landmark:

$$V_i \;=\; \{\, x \;:\; \mathrm{dist}(x, \ell_i) \leq \mathrm{dist}(x, \ell_{i'}) \text{ for all } i' \,\}.$$

The cells partition the space, so every query belongs to exactly one cell (ties are broken arbitrarily); they play the role of histogram buckets. Clustering places more landmarks, and hence smaller cells, where the corpus is dense. Each model has one landmark set per modality, and a query is located among the landmarks of the modality it searches: in text-to-image search, a text query is located among

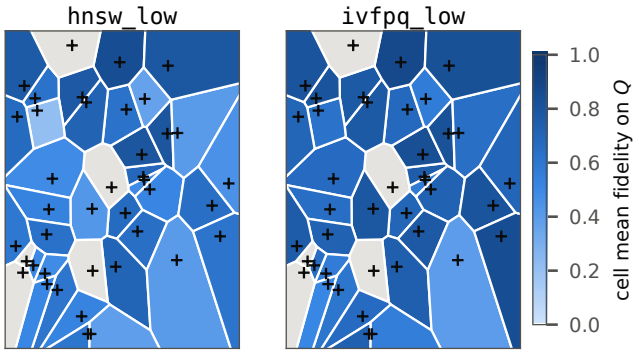


Figure 2: The offline stage on real data (Jina-v4, text-to-image, $K = 10{,}000$): 40 neighboring image landmarks (crosses), projected to two dimensions, with each cell shaded by the mean fidelity of `hnsw_low` (left) and `ivfpq_low` (right) on the construction sample; grey: sparse cells. The cells are the same in both panels, but the fidelity in them is not.

image landmarks. All configurations share the same landmarks, so the cells are the same for every configuration and only the statistics differ.

Step 2: Fidelity labels. For every query $q \in Q$, we compute its exact top-k once, run every configuration A_j on it, and record $y_{qj} = \mathrm{F}(q, A_j)$. This gives a $|Q| \times m$ table of true fidelities. It is the only step that uses exact search, once per sample query, since all configurations are compared against the same exact top-k.

Step 3: Assignment. Each sample query is assigned to the cell of its nearest landmark,

$$\ell(q) \;=\; \arg\min_{i}\; \mathrm{dist}(q, \ell_i), \qquad Q_i \;=\; \{\, q \in Q : \ell(q) = i \,\},$$

and $n_i = |Q_i|$ is the number of sample queries in cell i.

Step 4: Aggregation. For each cell i and configuration A_j, we keep the mean fidelity of the cell's sample queries,

$$\mu_{ij} \;=\; \frac{1}{n_i} \sum_{q \in Q_i} y_{qj},$$

and a histogram of their fidelities over B bins, from which we read, for any target τ, the fraction of the cell's queries on which A_j met it:

$$\pi_{ij}(\tau) \;=\; \frac{1}{n_i} \left|\{\, q \in Q_i : y_{qj} \geq \tau \,\}\right|.$$

We call $\pi_{ij}(\tau)$ the *coverage* of A_j in cell i: the estimated probability that A_j suffices for a query in this region. The mean says how good a configuration is on average; the coverage says how often it is good enough, which is what selection needs. Two cells with the same mean can differ sharply in coverage, if fidelity is uniformly moderate in one and high for most queries but very low for a few in the other. We use $B = 20$ bins of width 0.05; for a target at a bin edge, such as 0.7, 0.8, or 0.9, the coverage read from the histogram is exact, and for a target inside a bin, the bin's count is divided evenly among the fidelity values it holds.

Sparse cells. A cell with few or no sample queries has unreliable or undefined statistics. If fewer than $q_{\min}$ sample queries fall into cell i, its statistics are replaced by those of the whole sample, $\bar{\mu}_j$ and $\bar{\pi}_j(\tau)$. We use $q_{\min} = 10$.

Output. The synopsis consists of the K landmarks, the count n_i of each cell, and, for each cell and configuration, the mean μ_{ij} and a B-bin histogram: a $K \times m$ table with one row per cell and one column per configuration. Its size is $O(K(d + mB))$ numbers for d-dimensional embeddings, dominated by the landmarks themselves, and it grows neither with the corpus nor with the sample. The label table of Step 2 is needed only to build the synopsis.

3.3 Online Stage: Selecting a Configuration

At query time, the synopsis chooses a configuration for a new query q under the operator's target τ and reliability level ρ. Because each cell stores a histogram rather than a single number, the same synopsis serves any target.

Step 1: Locate the query. We find the query's nearest landmark, $i = \ell(q)$, by comparing q with all K landmarks, at a cost of $O(Kd)$.

Step 2: Read the estimates. Row i of the synopsis gives, for every configuration at once, an estimated fidelity and an estimated coverage:

$$\hat{F}(q, A_j) = \mu_{ij}, \qquad \hat{\pi}(q, A_j; \tau) = \pi_{ij}(\tau), \qquad j = 1, \ldots, m,$$

with the whole-sample statistics in place of μ_{ij} and $\pi_{ij}(\tau)$ if the cell is sparse. No configuration has been run on q at this point.

Step 3: Select. The *budget rule* prices each configuration by its cost plus a charge for its estimated chance of missing the target,

$$\hat{A}(q) \;=\; \arg\min_{A_j \in \mathcal{A}}\; c(A_j) + \lambda \left(1 - \hat{\pi}(q, A_j; \tau)\right).$$

The single parameter λ is the price of an expected miss. A small λ favors cheap configurations and lets violations rise; a large one favors reliable configurations. We set λ once per target, offline, to the smallest value whose violation rate on the construction sample meets the requirement, at most $1 - \rho$ (Section 4.5). Since the estimates depend only on the cell, the rule's choice is fixed per cell and can be precomputed, so Step 3 reduces to a table lookup.

The rule generalizes static deployment. With a single landmark ($K = 1$), every query falls into the same cell and receives the same configuration. This is a static plan, but not necessarily A_{static} of Section 2.4. As λ grows, a single cell's choice moves along the lower convex hull of the configurations' (violation rate, cost) points, so a configuration inside the hull is never chosen, and the grid of λ values is discrete. The penalized rule can therefore pick a costlier configuration than the constrained definition would. With more landmarks, the choice varies across cells: cells where cheap configurations suffice are served cheaply, and cells where they do not are served by costlier ones. The saving over the static plan comes entirely from this variation.

Simpler rules. Section 5.4 compares the budget rule with two simpler ones. The *mean rule* picks the cheapest configuration whose estimated fidelity $\hat{F}$ reaches τ, and the one with the highest estimate if none does; it ignores how fidelity is spread within a cell. The *coverage rule* picks the cheapest configuration whose estimated

coverage $\hat{\pi}$ reaches ρ in the query's cell, and the one with the highest coverage if none does; it enforces the requirement in every cell rather than over the workload, which is stricter and therefore costlier.

Step 4: Execute. We run $\hat{A}(q)$ and return its top-k result. The per-query overhead is one nearest-landmark search and one table lookup, independent of the corpus size, with no exact search.

4 EVALUATION SETUP

4.1 Benchmark

We use image–caption pairs from LAION [38]. Of the 6,999,488 pairs available to us, we remove 307 with a degenerate side (267 images with a side shorter than 16 pixels, mostly thin strips and banners, and 40 captions that are missing, unreadable, or one to two characters long), and then 172,894 duplicates found by grouping near-identical SigLIP2 embeddings: 147,566 pairs whose image belongs to a large group of identical images under mostly unrelated captions, which we take to be a placeholder for an image that failed to download, and 25,328 extra copies of pairs whose image and caption both repeat, at least one of them in a large group; of each such pair we keep one copy. Duplicates make a query's exact top-100 arbitrary, since tied items can be returned in any order; we remove the large groups (the largest 5% by size: eight or more identical images, or six or more identical captions), while smaller groups of repeated captions remain, so some queries still have ties at rank 100. The remaining 6,826,287 pairs are split once into three disjoint sets in proportions 85/10/5%, stratified on the 616 non-empty combinations of two k-means clusterings with 25 clusters each, one of the image and one of the caption embeddings (seed 0):

- the *corpus* C (5,802,772 pairs), which is indexed;
- the *construction sample* Q (682,444 pairs), used only offline, to build the synopses, choose the static plan, and tune λ;
- the *held-out workload* Q_{test} (341,071 pairs), on which all results are reported.

The image and the caption of a pair always belong to the same set, so no query shares a pair with an item it searches. The split is computed in SigLIP2's embedding space and reused for the other models, so that all models are evaluated on the same pairs. Each model indexes the images and the captions of C as two separate collections, and every pair in Q and Q_{test} provides an image query and a text query, each searched against both collections. This gives the four query directions, with 341,071 held-out queries per setting. Unless stated otherwise, $k = 100$.

4.2 Embedding Models

We use four open-weight embedding models (Table 3), chosen to differ in properties that affect approximate search: their architecture (SigLIP2 [42] and OpenCLIP [8] are contrastively trained dual encoders, Jina-v4 [11] and Qwen3-VL [26] embedders built on vision–language models), their dimension, which affects the cost of every configuration and of the synopsis lookup, and how strongly they separate the two modalities, which determines how far cross-modal queries lie from the corpus they search [27]. Every structure (indexes, exact neighbors, landmarks, synopses) is built separately for each model; using four models tests whether our conclusions hold across independently built embedding spaces.

Model	Checkpoint	Type	d	Tokens	Precision
SigLIP2	`google/siglip2-so400m-patch16-512`	dual encoder	1,152	64	fp16
OpenCLIP	`ViT-L-14, laion2b_s32b_b82k`	dual encoder	768	77	fp16
Jina-v4	`jinaai/jina-embeddings-v4`	VLM (Qwen2.5-VL)	2,048	512	bf16
Qwen3-VL	`Qwen/Qwen3-VL-Embedding-2B`	VLM (Qwen3-VL)	2,048	512	bf16

Table 3: Embedding models. d: embedding dimension; Tokens: maximum caption length; Precision: inference precision.

4.3 Configuration Catalog

For each model, we build three configurations of each index family, *low*, *moderate*, and *high*, giving nine configurations (Table 4), with FAISS [17] 1.15.1 (official cuVS build, with AVX2). The three tiers of a family differ in their build: HNSW in graph degree and `efConstruction`, the IVF families in code length. They were chosen by a parameter sweep on a 50,000-vector sample to reach mean fidelity bands of 0.65–0.75 (low), 0.75–0.85 (moderate), and at least 0.85 (high), averaged over the two directions an index serves; the search parameter was then set per model at full scale, on 5,000 construction queries. At full scale every IVF-family tier lies in its band, but `hnsw_low` lies below its band for every model. Three tiers per family give a choice between cheap and expensive configurations both within and across families, while keeping the catalog small enough to run every configuration on every construction query.

Reaching the bands requires `nprobe` values of 2,048–4,096 for SigLIP2 and OpenCLIP but only 16–256 for Jina-v4 and Qwen3-VL. For SigLIP2 and OpenCLIP, every IVF-family configuration is therefore several times as expensive as `hnsw_low`, and most of them are dominated; for Jina-v4 and Qwen3-VL, the low and moderate IVF tiers cost between `hnsw_low` and `hnsw_mod`. No configuration is removed, since a dominated configuration can still be the cheapest sufficient one for individual queries. The parameters are what the sweep selected to reach each band. They were not tuned per model for cost, and some look unusual. The high `nprobe` values for SigLIP2 and OpenCLIP are one example. The build of `hnsw_high` also uses a smaller efConstruction than that of `hnsw_mod`. A differently tuned configuration could change the static plan in some settings. We therefore do not rest the result on any single configuration. Section 5.4.1 compares against the best static choice among eleven configurations, including two cheaper search settings of the `hnsw_high` index, and the saving persists. Table 5 gives the cost and mean fidelity of every configuration in every setting.

4.4 Cost Measurement

All costs are measured on dedicated nodes with an AMD EPYC 7763 processor (128 logical CPUs, 64 used) and 1 TB of RAM, with all indexes in memory; exact search and k-means run on NVIDIA A100-SXM4-80GB GPUs. We measure two regimes: *single query*, one query at a time on one thread, and *batched*, batches of 4,096 queries on 64 threads. Every measurement uses the same 5,000 queries drawn at random from Q_{test}, after an untimed warm-up of 500 queries. In the batched regime, passes over the sample are repeated until at least 20 s and three passes have been timed, and we report the median pass; in the single-query regime, we report

Configuration	Build parameters	Search parameter	Shared	Memory (GB)
hnsw_low	M=4, efC=40	efSearch=1024/1024/192/128	no	54.1/36.3/95.7/95.7
hnsw_mod	M=8, efC=80	efSearch=1536/1024/192/256	no	54.5/36.7/96.1/96.1
hnsw_high	M=16, efC=40	efSearch=6144/6144/512/1024	no	55.2/37.4/96.8/96.8
ivfpq_low	n_{list}=32768, PQ288/192/512/512	nprobe=2048/2048/16/16	no	3.7/2.5/6.6/6.6
ivfpq_mod	n_{list}=32768, PQ384/256/1024/1024	nprobe=2048/2048/32/32	no	4.9/3.3/12.5/12.5
ivfpq_high	n_{list}=32768, PQ1152/768/2048/2048	nprobe=4096/4096/128/256	no	13.8/9.2/24.4/24.4
oivfpq_low	OPQ288/192/512/512, n_{list}=32768, PQ288/192/512/512	nprobe=2048/2048/16/16	no	3.8/2.5/6.6/6.6
oivfpq_mod	OPQ384/256/1024/1024, n_{list}=32768, PQ384/256/1024/1024	nprobe=2048/2048/32/32	no	4.9/3.3/12.6/12.6
oivfpq_high	OPQ1152/768/2048/2048, n_{list}=32768, PQ1152/768/2048/2048	nprobe=4096/4096/128/256	no	13.8/9.2/24.4/24.4

Table 4: The nine configurations of the catalog, with per-model values in the order SigLIP2/OpenCLIP/Jina-v4/Qwen3-VL. Every configuration is a separate build; memory is that of the image and the text index together. Table 5 gives each configuration's cost and mean fidelity per setting.

the mean and the 99th percentile. The cost $c(A)$ of a configuration is its mean per-query latency for the given model and direction. The lookup cost c_{est} is timed with the same harness, queries, batch size, and number of threads, in a separate run on the same type of node. All headline results use the batched regime. Costs assume that every index receives full batches. Under per-query selection, a batch is split across the indexes the synopsis chooses, so each index sees only part of it. A service under sustained load can accumulate the queries for each index into full batches, at some cost in waiting time. We do not model this effect.

4.5 Synopsis Parameters and Reference Points

Synopsis parameters. Unless stated otherwise, each modality has K = 10,000 landmarks; RQ5 varies K. Histograms have $B = 20$ bins, and $q_{\min} = 10$. We evaluate fidelity targets $\tau \in \{0.7, 0.8, 0.9\}$, with $\tau = 0.7$ as the default, and a reliability level of $\rho = 0.9$, so the requirement allows at most 10% of queries to miss the target.

Choice of target. A target of $\tau = 0.7$ at $k = 100$ asks a configuration to return 70 of a query's 100 exact nearest neighbors. Fidelity is measured against exact search, not against relevance. The exact neighbors depend on the embedding model and are only a proxy for what a user would judge relevant. The catalog's tiers span mean fidelities from about 0.65 to above 0.9 (Table 5). At $\tau = 0.7$, every model therefore has a real choice between cheap and costly configurations. At $\tau = 0.9$, many queries miss the target under every configuration, so the requirement can be met in only 9 of 16 settings, even with per-query choice. This is mostly a limit of the catalog, not of the synopsis. At $k = 10$, mean fidelity differs from that at $k = 100$ by −0.04 to +0.08 for every configuration and setting, and the configurations rank almost identically (Table 7). Location predicts fidelity less well at $k = 10$ (Table 6). Skill, the ceiling, ρ_s, Brier skill, and AUC are lower in every setting, with a median skill of 0.23 against 0.47. The ceiling falls just as much, from 0.49 to 0.24, so the synopsis still captures nearly all that its cells allow. The drop is likely due to the coarser measure. Fidelity at $k = 10$ takes only 11 values and so varies more between queries in the same cell. Calibration stays good (ECE below 0.008 in every setting, against below 0.006 at $k = 100$), and fewer queries are unservable in 11 of the 16 settings. We have not evaluated plan selection at $k = 10$.

We fix $\rho = 0.9$ throughout. The trade-off curves of Section 5.4 cover every other reliability level. Section 5.4 reports plan selection at all three targets (Table 14).

Tuning λ. The budget rule's λ is chosen on Q by 5-fold cross-fitting: the synopsis is built on four folds and its violation rate measured on the fifth, and we take the smallest λ whose out-of-fold violation rate meets the chosen limit; the final synopsis is then built on all of Q. We use two limits: the requirement, at most $1 - \rho$ (*budget rule at the requirement*, our method), and the violation rate of the static plan on Q (*budget rule at the static rate*, the conservative comparison of Section 2.4). If no λ reaches the limit, we use the one with the fewest violations.

Reference points. We compare the synopsis with location-blind estimates (the *global average* fidelity and the *global coverage* of each configuration on Q, which equal a synopsis with $K = 1$), with a *density-only* estimate (RQ2), with a *shared synopsis* that keeps one difficulty value per cell for all configurations (RQ3), with the static plan, and with three versions of the per-query optimum. The *matched optimum* serves the same queries as the static plan and possibly more: it runs every query whose cheapest sufficient configuration costs no more than the static plan, and charges every other query the static plan's cost. The *optimum at the requirement* is the cheapest assignment of configurations that misses at most $1 - \rho$ of the queries; no selector that meets the requirement can be cheaper. The *unconstrained optimum* serves every query that some configuration can serve. Table 8 defines each precisely.

4.6 Metrics

All metrics are computed on Q_{test}, for synopses, static plans, and λ fitted on Q. Estimation metrics are computed per configuration and averaged over configurations, excluding *flat* configurations, whose true fidelity has a standard deviation below 0.03 in a direction, since skill is not meaningful on a near-constant target. Table 8 summarizes all metrics and the abbreviations used in the tables; we define the main ones here.

		SigLIP2		OpenCLIP		Jina-v4		Qwen3-VL	
Dir.	Configuration	Cost	$\bar{F}$	Cost	$\bar{F}$	Cost	$\bar{F}$	Cost	$\bar{F}$
i→i	hnsw_low	651	0.683	484	0.674	208	0.728	160	0.656
	hnsw_mod	1,609	0.964	878	0.950	451	0.936	527	0.949
	hnsw_high	13,439	0.978	8,796	0.973	1,141	0.978	1,976	0.981
	ivfpq_low	3,582	0.890	3,050	0.874	297	0.817	313	0.815
	ivfpq_mod	18,365	0.935	12,263	0.923	420	0.899	399	0.895
	ivfpq_high	23,743	0.992	14,833	0.989	1,855	0.969	3,549	0.976
	oivfpq_low	3,455	0.889	2,341	0.874	323	0.819	353	0.816
	oivfpq_mod	18,251	0.935	12,194	0.924	428	0.899	407	0.895
	oivfpq_high	23,468	0.993	14,818	0.991	2,023	0.969	3,842	0.977
i→t	hnsw_low	647	0.311	494	0.358	234	0.601	171	0.468
	hnsw_mod	1,579	0.748	961	0.714	525	0.868	581	0.869
	hnsw_high	13,145	0.831	8,560	0.744	1,384	0.941	1,870	0.940
	ivfpq_low	2,958	0.486	2,247	0.540	309	0.703	311	0.729
	ivfpq_mod	17,648	0.644	12,106	0.714	397	0.809	397	0.834
	ivfpq_high	19,339	0.862	13,854	0.926	1,813	0.925	3,848	0.958
	oivfpq_low	3,027	0.493	2,232	0.562	302	0.702	337	0.729
	oivfpq_mod	17,609	0.653	12,040	0.727	403	0.809	393	0.835
	oivfpq_high	19,233	0.865	13,786	0.929	2,008	0.925	3,483	0.959
t→i	hnsw_low	673	0.490	531	0.525	228	0.562	146	0.386
	hnsw_mod	1,735	0.871	923	0.860	506	0.823	532	0.733
	hnsw_high	13,150	0.933	8,646	0.939	1,130	0.915	2,071	0.897
	ivfpq_low	3,116	0.468	2,018	0.493	320	0.670	337	0.530
	ivfpq_mod	17,755	0.626	11,725	0.645	384	0.782	384	0.648
	ivfpq_high	20,083	0.876	12,335	0.899	1,703	0.907	3,369	0.871
	oivfpq_low	2,991	0.477	1,972	0.512	302	0.672	322	0.531
	oivfpq_mod	17,693	0.634	11,741	0.654	418	0.782	395	0.648
	oivfpq_high	19,983	0.878	12,133	0.901	1,857	0.907	3,780	0.871
t→t	hnsw_low	688	0.500	450	0.470	221	0.611	176	0.522
	hnsw_mod	1,687	0.912	971	0.842	501	0.886	602	0.905
	hnsw_high	14,220	0.929	9,146	0.809	1,226	0.937	2,192	0.953
	ivfpq_low	3,751	0.878	2,473	0.878	311	0.765	306	0.770
	ivfpq_mod	18,735	0.926	12,329	0.926	391	0.849	404	0.850
	ivfpq_high	27,356	0.988	16,032	0.988	1,779	0.931	4,010	0.950
	oivfpq_low	3,865	0.879	2,449	0.879	316	0.764	323	0.770
	oivfpq_mod	18,638	0.927	12,338	0.927	400	0.849	412	0.850
	oivfpq_high	27,188	0.989	15,812	0.989	1,874	0.931	3,737	0.950

Table 5: Per-query cost (μs per query; controlled benchmark, batches of 4,096 on 64 threads) and mean fidelity $\bar{F}$ on Q_{test} of every configuration, per model and direction.

Accuracy of estimates. For the estimated fidelity, we report the mean absolute error and the *MSE skill score* [32],

$$\text{Skill} \;=\; 1 - \frac{\text{MSE}(\hat{F})}{\text{MSE}(\text{global average})},$$

which is 0 for an estimate no better than the global average, 1 for a perfect one, and negative for a worse one. For the estimated coverage, a predicted probability of the event $\text{F}(q, A_j) \geq \tau$, we report the *Brier skill score* [10], the same construction applied to the mean squared error of that probability against the global coverage; the AUC for separating queries that meet the target from those that do not; and the expected calibration error (ECE) [12, 33], over 10 equal-width bins of $\hat{\pi}$, restricted to pairs of query and configuration with $0.05 < \hat{\pi} < 0.95$, since ECE over all pairs is dominated by configurations whose coverage is trivially 0 or 1.

Plan selection. For a selector, we report its *relative cost*, the mean of $c_{\text{est}} + c(\hat{A}(q))$ divided by the cost of the static plan; its *violation rate*; and the share of the available saving it captures, the *savings recovered*,

$$\frac{c(A_{\text{static}}) - \bar{c}_{\text{selector}}}{c(A_{\text{static}}) - \bar{c}_{\text{opt}}},$$

which is 0 for no improvement over the static plan and 1 for a selector as cheap as the corresponding optimum, and is reported only where that optimum is at least 5% cheaper than the static plan. A selector can exceed 1 only by violating more often than the optimum allows; in our results, only selectors that violate more often than the static plan do.

Metric ($\tau = 0.7$, $K = 10{,}000$)	Median $k = 10$	Median $k = 100$	$k = 10$ better
Skill	0.226	0.468	0 of 16
ICC (ceiling)	0.236	0.487	0 of 16
Spearman ρ_s	0.392	0.662	0 of 16
Brier skill (coverage)	0.131	0.236	0 of 16
AUC	0.812	0.866	0 of 16
ECE	0.005	0.004	2 of 16
Unservable queries	0.3%	0.5%	fewer in 11, more in 5
Gap to ceiling (ICC − skill), median	**0.012**	**0.010**	

Table 6: Accuracy of the synopsis when fidelity is measured at $k = 10$ instead of $k = 100$, with medians over the 16 settings. Every metric but ECE is lower at $k = 10$ in every setting, but so is the ceiling, and the synopsis stays about as close to it (bold: the median over the settings of each setting's gap). Unservable: held-out queries that no configuration serves at $\tau = 0.7$. Terms: Table 8.

4.7 Statistical Analysis

With 341,071 held-out queries per setting, almost any difference is statistically significant, so we lead with effect sizes and confidence intervals and use hypothesis tests as supporting evidence. We compute 95% bootstrap confidence intervals with 1,000 resamples; since queries in the same cell share their estimate, we resample cells rather than individual queries. Each comparison uses a paired test suited to its outcome: the Wilcoxon signed-rank test on per-query squared error for estimation error (synopsis against global average in RQ2, keyed against shared synopsis in RQ3), the same test with Pratt's handling of zero differences for shortfall (RQ4), and McNemar's test on per-query outcomes for violations (RQ4). Within each family of tests, p-values are corrected across settings with the Benjamini–Hochberg procedure at a false discovery rate of $q < 0.05$.

Robustness. Over three seeds of the landmark clustering, at $K = 10{,}000$ and $\tau = 0.7$, the standard deviation of skill, Brier skill, relative cost, and violation rate is small in every setting (at most 0.008 for skill and 0.010 for Brier skill, 0.03 for relative cost, and a quarter of a percentage point for violation rates), so no conclusion depends on the seed.

5 EXPERIMENTS

We answer five research questions. RQ1 measures the opportunity; RQ2 and RQ3 test the synopsis's two premises; RQ4 evaluates plan selection end to end; RQ5 measures the cost of the synopsis and its sensitivity to its parameters.

5.1 RQ1: How much can per-query plan selection save?

Before evaluating the synopsis, we measure the opportunity it targets: how much cheaper a workload would be if every query ran on its cheapest sufficient configuration. Table 9 reports, for each setting at $\tau = 0.7$, the static plan, how the held-out queries are distributed by the cost of their cheapest sufficient configuration, and the cost of the per-query optimum. We use the matched optimum, which serves at least every query the static plan serves and is therefore the like-for-like comparison; the unconstrained optimum also pays for queries the static plan misses, and can cost more.

The opportunity is large in most settings. At matched service, per-query selection would lower cost in every setting, and to about half of the static plan in a typical one. The settings fall into three groups (bold and grey in Table 9). In seven cross-modal settings, the opportunity is large: the static plan is the most accurate tier of a family, yet most queries would meet the target on a configuration costing at most a third as much. In seven settings, the opportunity is moderate: most queries could use a cheaper configuration, mostly one costing between a third and the whole of the static plan. In two settings, it is small: the static plan is `hnsw_mod`, and its only cheaper alternative, `hnsw_low`, suffices for only part of the queries.

The hardest queries set the price for all of them. The size of the opportunity follows from two quantities: how expensive the static plan is, and how many queries actually need it. The static plan must serve a fraction ρ of the workload, so it is sized for the hardest queries within that fraction. Where these force `hnsw_high`, which costs several times as much as `hnsw_mod`, every query pays for them, although in the large-opportunity settings fewer than a fifth of queries need the static plan's configuration. Conversely, where most queries do need it, as in SigLIP2 t→t, the opportunity is small. Every setting with a hard tail of this kind is cross-modal, consistent with cross-modal queries lying away from the corpus distribution; the only cross-modal setting without one is Qwen3-VL i→t.

Which configurations provide the saving. Figure 3 breaks the held-out queries down by their cheapest sufficient configuration. For SigLIP2 and OpenCLIP, it is usually an HNSW configuration: their IVF-family configurations probe thousands of partitions and are the cheapest choice mainly where HNSW falls short, as in image-to-text search. For Jina-v4 and Qwen3-VL, whose low and moderate IVF tiers are cheaper than `hnsw_mod`, an IVF-family configuration is the cheapest sufficient one for a quarter to two thirds of the queries. Since every configuration in this catalog is a separate index, capturing the saving requires keeping several indexes built; Section 5.4.1 examines how much of it a smaller catalog retains.

Reliability as well as cost. Per-query selection can also serve queries that the static plan misses: up to a tenth of queries (column "> 1×") miss the target under the static plan but meet it under a costlier configuration. The share is largest for Jina-v4 and Qwen3-VL i→i, whose static plan only just meets the requirement. Queries that no configuration can serve are rare, at most 5% in any setting.

The static plan generalizes. The static plan, chosen on Q, meets the target on at least 90% of the held-out queries in every setting where some configuration meets it on 90% of Q. The baseline is therefore stable across samples, and the opportunity is not an artifact of evaluating a plan on the queries it was tuned on.

Other targets. At $\tau = 0.8$ and 0.9, the matched optimum is again at least 5% cheaper than the static plan in every setting, with a median close to half of it.

Answer to RQ1. At the same level of service, per-query configuration choice would roughly halve the cost of a static deployment in a typical setting, and save most where the static plan is sized for

	SigLIP2				OpenCLIP			
Configuration	i→i	i→t	t→i	t→t	i→i	i→t	t→i	t→t
hnsw_low	0.693 (0.683)	0.324 (0.311)	0.500 (0.490)	0.508 (0.500)	0.684 (0.674)	0.391 (0.358)	0.543 (0.525)	0.488 (0.470)
hnsw_mod	0.968 (0.964)	0.791 (0.748)	0.892 (0.871)	0.916 (0.912)	0.958 (0.950)	0.763 (0.714)	0.889 (0.860)	0.852 (0.842)
hnsw_high	0.981 (0.978)	0.855 (0.831)	0.942 (0.933)	0.933 (0.929)	0.977 (0.973)	0.778 (0.744)	0.950 (0.939)	0.824 (0.809)
ivfpq_low	0.886 (0.890)	0.461 (0.486)	0.459 (0.468)	0.881 (0.878)	0.866 (0.874)	0.502 (0.540)	0.491 (0.493)	0.881 (0.878)
ivfpq_mod	0.928 (0.935)	0.638 (0.644)	0.629 (0.626)	0.924 (0.926)	0.913 (0.923)	0.683 (0.714)	0.649 (0.645)	0.922 (0.926)
ivfpq_high	0.991 (0.992)	0.894 (0.862)	0.909 (0.876)	0.986 (0.988)	0.988 (0.989)	0.927 (0.926)	0.920 (0.899)	0.985 (0.988)
oivfpq_low	0.886 (0.889)	0.464 (0.493)	0.472 (0.477)	0.883 (0.879)	0.867 (0.874)	0.526 (0.562)	0.514 (0.512)	0.882 (0.879)
oivfpq_mod	0.929 (0.935)	0.645 (0.653)	0.640 (0.634)	0.924 (0.927)	0.914 (0.924)	0.698 (0.727)	0.662 (0.654)	0.923 (0.927)
oivfpq_high	0.992 (0.993)	0.898 (0.865)	0.915 (0.878)	0.987 (0.989)	0.990 (0.991)	0.935 (0.929)	0.925 (0.901)	0.987 (0.989)

	Jina-v4				Qwen3-VL			
Configuration	i→i	i→t	t→i	t→t	i→i	i→t	t→i	t→t
hnsw_low	**0.780 (0.728)**	**0.682 (0.601)**	0.600 (0.562)	**0.664 (0.611)**	**0.723 (0.656)**	**0.530 (0.468)**	0.413 (0.386)	**0.582 (0.522)**
hnsw_mod	0.965 (0.936)	**0.929 (0.868)**	0.867 (0.823)	0.923 (0.886)	0.972 (0.949)	0.917 (0.869)	0.761 (0.733)	0.936 (0.905)
hnsw_high	0.985 (0.978)	0.962 (0.941)	0.929 (0.915)	0.951 (0.937)	0.986 (0.981)	0.955 (0.940)	0.903 (0.897)	0.963 (0.953)
ivfpq_low	0.851 (0.817)	**0.761 (0.703)**	0.711 (0.670)	0.814 (0.765)	0.850 (0.815)	0.766 (0.729)	**0.586 (0.530)**	0.819 (0.770)
ivfpq_mod	0.931 (0.899)	**0.872 (0.809)**	0.829 (0.782)	0.895 (0.849)	0.928 (0.895)	0.882 (0.834)	**0.720 (0.648)**	0.896 (0.850)
ivfpq_high	0.981 (0.969)	0.957 (0.925)	0.932 (0.907)	0.956 (0.931)	0.984 (0.976)	0.972 (0.958)	0.906 (0.871)	0.967 (0.950)
oivfpq_low	0.853 (0.819)	**0.760 (0.702)**	0.713 (0.672)	0.813 (0.764)	0.851 (0.816)	0.765 (0.729)	**0.586 (0.531)**	0.819 (0.770)
oivfpq_mod	0.932 (0.899)	**0.872 (0.809)**	0.830 (0.782)	0.895 (0.849)	0.928 (0.895)	0.882 (0.835)	**0.720 (0.648)**	0.896 (0.850)
oivfpq_high	0.981 (0.969)	0.958 (0.925)	0.933 (0.907)	0.956 (0.931)	0.985 (0.977)	0.972 (0.959)	0.906 (0.871)	0.967 (0.950)

Table 7: Mean fidelity of every configuration at $k = 10$, with $k = 100$ in parentheses, on the held-out queries. The two differ by -0.04 to $+0.08$, and the configurations rank almost identically at both depths. Bold: fidelity at $k = 10$ higher by at least 0.05.

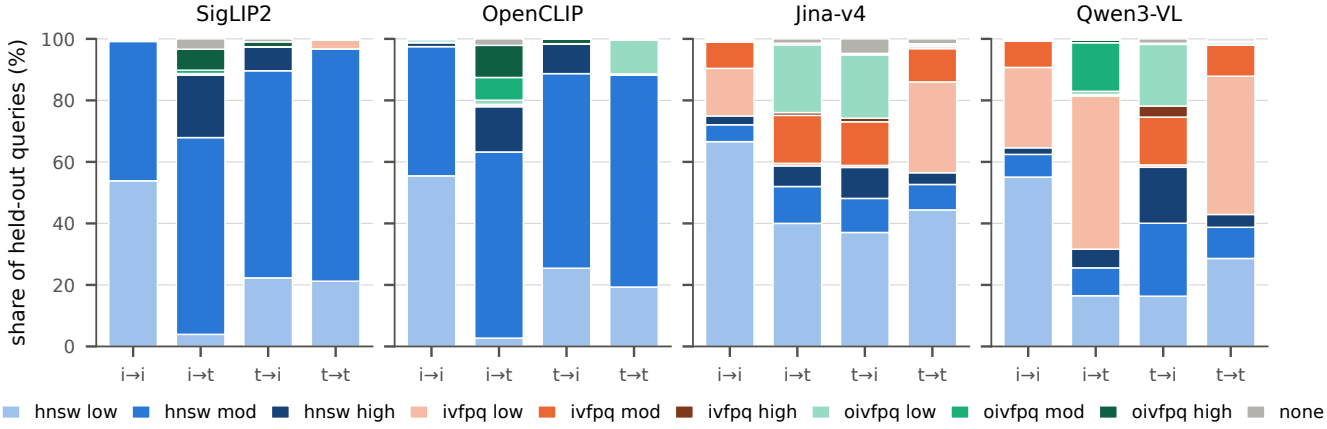


Figure 3: Held-out queries by their cheapest sufficient configuration ($\tau = 0.7$); none: no configuration meets the target. Shades: low, moderate, and high tier of each family.

a small hard tail of cross-modal queries. Whether this opportunity can be captured without knowing each query's fidelity in advance is the subject of RQ2–RQ4.

5.2 RQ2: Does a query's location predict its fidelity?

The smoothness premise says that queries in the same cell have similar fidelity under a given configuration. The synopsis provides two estimates per configuration, the cell mean $\hat{F}$ and the coverage $\hat{\pi}$; selection uses the coverage, while the mean is the most direct test of

Term (abbreviation)	Meaning
Workload and baselines	
Setting	A pair of embedding model and query direction; 16 in total.
Requirement	At most a fraction $1 - \rho = 10\%$ of queries may miss the fidelity target τ.
Static plan	The cheapest single configuration that meets τ on at least a fraction ρ of Q; if none does, the one that meets it most often (*, §).
Serves	Share of held-out queries on which a configuration meets τ.
Cheapest sufficient configuration	The cheapest configuration that meets τ on a given query; buckets give its cost relative to the static plan (none: no configuration meets τ).
Optimum, matched	Runs every query whose cheapest sufficient configuration costs no more than the static plan on that configuration, and charges every other query the cost of the static plan; it serves at least every query that the static plan serves.
Optimum, unconstrained (all)	Runs every query that some configuration can serve on its cheapest sufficient configuration, and every other query on its highest-fidelity configuration.
Optimum, $\leq 1-\rho$	The cheapest assignment of configurations that misses τ on at most $1 - \rho$ of the held-out queries, a missed query running on the cheapest configuration; no selector that meets the requirement is cheaper (infeas.: more than $1 - \rho$ of queries cannot be served by any configuration).
Selection rules (all use the synopsis's estimates for the query's cell)	
Mean rule	The cheapest configuration whose estimated fidelity $\hat{F}$ reaches τ.
Coverage rule	The cheapest configuration whose estimated coverage $\hat{\pi}$ reaches ρ; if none does, the one with the highest coverage.
Budget rule	$\arg\min_j c(A_j) + \lambda(1 - \hat{\pi}_j)$, with λ tuned on Q to a violation limit: the requirement, $1 - \rho$ (*at the requirement*, $\leq 1-\rho$; our method), or the static plan's violation rate on Q (*at the static rate*, static viol.); ‡: limit not reachable, fewest violations used.
Selection metrics (on Q_{test})	
Cost	Mean of $c_{\text{est}} + c(\hat{A}(q))$, relative to the static plan (1×).
Violation rate (Viol.)	Share of queries whose chosen configuration misses τ.
Shortfall (Short.)	Mean of $\tau - \mathrm{F}(q, \hat{A}(q))$ over violated queries.
Savings recovered (Rec.)	Share of the optimum's saving over the static plan that a selector captures: against the matched optimum, except for the budget rule at the requirement, measured against the optimum at $\leq 1-\rho$; shown where that optimum saves at least 5%.
†	The coverage rule's violated queries fall less far below τ than the mean rule's (mean shortfall over violated queries).
Estimation metrics (on Q_{test}, averaged over non-flat configurations)	
Non-flat configuration	True fidelity has a standard deviation of at least 0.03 in the direction.
Global average (GA), global coverage	Location-blind estimates: a configuration's mean fidelity, or share of queries meeting τ, on all of Q; equivalent to a synopsis with $K = 1$.
Density-only (DO)	Global average plus a linear term in the local density of the query's cell (mean distance from its landmark to its 100 nearest corpus vectors), with one slope per configuration (per-cfg) or one for all (shared).
Skill	MSE skill score of the estimated fidelity against the global average: 0 no better, 1 perfect.
ρ_s	Spearman rank correlation between estimated and true fidelity.
ICC	Share of fidelity variance that lies between cells on Q_{test}; approximately the highest skill any estimate with one value per cell can reach.
Interior (Int.), boundary (Bnd.)	Skill on the third of queries with the lowest, or highest, ratio of distances to the nearest and second-nearest landmark.
Brier skill (BSS)	Skill of the estimated coverage, as a probability of meeting τ, against the global coverage.
AUC, ECE	How well the estimated coverage separates queries that meet τ from those that do not; its expected calibration error over 10 bins, for $0.05 < \hat{\pi} < 0.95$.
Keyed, shared synopsis	The synopsis, with statistics per configuration; a variant with one difficulty offset per cell for all configurations.
Thin	Share of held-out queries in cells with fewer than $q_{\min}$ sample queries.
Configuration abbreviations	
H, I, O; l, m, h	HNSW, IVFPQ, O-IVFPQ; low, moderate, high tier. In the single index, l, m, h are its three `efSearch` settings, and in the union, S-l and S-m are its two lower ones.

Table 8: Ready reckoner: the terms, metrics, and abbreviations used in the tables of Section 5.

whether location carries information about fidelity at all. Table 10 evaluates both on every held-out query, against the location-blind global average and global coverage.

Location carries information in every setting. The synopsis is more accurate than the global average in every setting, with skill well above zero everywhere (all confidence intervals exclude zero, and every per-configuration Wilcoxon test is significant). Location alone explains roughly half of the variance in fidelity in a typical setting, and the synopsis orders queries by fidelity moderately well. Local density does not explain it: the density-only estimate has much lower skill than the synopsis in every setting, so the synopsis captures more than how crowded a region is.

The information is partial, however: the other half of the variance lies among queries within the same cell, where the synopsis cannot see it. Whether the synopsis nevertheless chooses configurations well is the question of RQ4.

The synopsis reaches almost all that its partition allows. The ICC column, the share of fidelity variance that lies between cells on the held-out queries themselves, approximates the highest skill any estimate with one value per cell could reach. The synopsis's skill, with cell means estimated on the construction sample, comes within a few hundredths of it in every setting. The cell statistics therefore carry over to new queries almost unchanged: the synopsis is limited not by its sample or by overfitting, but by how finely the space is partitioned and how much fidelity varies within a cell. Consistent with this, skill rises with the number of landmarks until cells become too thin (Section 5.5).

The coverage estimate is calibrated. The coverage that selection relies on is also informative: its Brier skill is positive in every setting,

Model	Dir.	Static plan	Static serves	Queries whose cheapest sufficient plan costs ≤ 0.33×	0.33–1×	1×	> 1×	none	Optimum cost all	matched
SigLIP2	i→i	hnsw_mod	99.2%	0.0%	53.8%	45.3%	0.8%	0.0%	0.72×	0.68×
	i→t	oivfpq_high*	89.9%	68.1%	21.6%	6.8%	0.1%	3.4%	0.30×	**0.31**×
	t→i	hnsw_high	97.4%	89.7%	0.0%	7.7%	1.6%	1.0%	0.21×	**0.20**×
	t→t	hnsw_mod	96.6%	0.0%	21.2%	75.4%	3.1%	0.2%	0.94×	0.87×
OpenCLIP	i→i	hnsw_mod	97.5%	0.0%	55.5%	42.0%	2.5%	0.0%	0.92×	0.75×
	i→t	oivfpq_high	97.2%	64.7%	22.7%	10.5%	0.1%	2.1%	0.33×	**0.33**×
	t→i	hnsw_high	98.4%	88.8%	0.0%	9.6%	1.5%	0.1%	0.20×	**0.20**×
	t→t	oivfpq_low	99.1%	19.3%	69.0%	11.2%	0.3%	0.2%	0.44×	0.43×
Jina-v4	i→i	ivfpq_mod	90.9%	0.0%	82.4%	8.6%	8.7%	0.2%	0.68×	0.62×
	i→t	hnsw_high	97.2%	79.0%	12.0%	6.7%	0.9%	1.4%	0.30×	**0.30**×
	t→i	hnsw_high	93.7%	58.3%	25.7%	10.1%	1.2%	4.7%	0.41×	**0.39**×
	t→t	hnsw_mod	91.6%	0.0%	85.7%	8.3%	4.4%	1.6%	0.71×	0.61×
Qwen3-VL	i→i	ivfpq_mod	90.1%	0.0%	81.6%	8.6%	9.8%	0.0%	0.73×	0.61×
	i→t	hnsw_mod	90.6%	16.5%	67.1%	9.1%	6.9%	0.4%	0.79×	0.60×
	t→i	hnsw_high	92.6%	76.8%	0.0%	18.2%	3.6%	1.4%	0.39×	**0.37**×
	t→t	hnsw_mod	93.6%	28.6%	56.2%	10.1%	4.4%	0.7%	0.67×	0.54×

Table 9: Opportunity for per-query selection at $\tau = 0.7$: the static plan and the share of held-out queries it serves; the held-out queries by the cost of their cheapest sufficient configuration, relative to the static plan; and the mean cost of the unconstrained and the matched optimum. Bold: large opportunity (matched optimum below 0.4×); grey: small (above 0.7×). Terms: Table 8.

Model	Dir.	MAE GA	MAE Synopsis	MAE Skill [95% CI]	DO skill per-cfg	DO skill shared	ρ_s	Skill ICC	Skill Int.	Skill Bnd.	Coverage $\hat{\pi}$, τ=0.7 BSS [95% CI]	AUC	ECE
SigLIP2	i→i	0.053	0.031	0.581 [0.574, 0.588]	0.348	0.294	0.728	0.591	**0.529**	0.512	0.199 [0.181, 0.216]	0.889	0.0028
	i→t	0.118	0.090	0.349 [0.320, 0.378]	0.033	0.030	0.609	0.357	**0.390**	0.274	0.219 [0.200, 0.237]	0.832	0.0021
	t→i	0.136	0.104	0.320 [0.310, 0.330]	0.017	0.004	0.583	0.331	**0.285**	0.251	0.204 [0.196, 0.212]	0.814	0.0018
	t→t	0.054	0.034	0.587 [0.466, 0.663]	0.059	0.032	0.632	0.595	**0.758**	0.284	0.493 [0.360, 0.574]	0.921	0.0053
OpenCLIP	i→i	0.062	0.034	0.594 [0.587, 0.601]	0.389	0.339	0.734	0.604	0.516	**0.529**	0.237 [0.226, 0.250]	0.907	0.0030
	i→t	0.090	0.067	0.386 [0.365, 0.407]	0.016	0.008	0.599	0.398	**0.374**	0.329	0.230 [0.218, 0.241]	0.840	0.0025
	t→i	0.120	0.090	0.364 [0.330, 0.392]	0.075	0.053	0.592	0.374	**0.382**	0.310	0.214 [0.195, 0.229]	0.830	0.0035
	t→t	0.067	0.043	0.578 [0.471, 0.641]	0.038	−0.029	0.597	0.584	**0.730**	0.261	0.503 [0.377, 0.564]	0.924	0.0027
Jina-v4	i→i	0.087	0.052	0.527 [0.518, 0.535]	0.342	0.275	0.725	0.535	0.281	**0.458**	0.273 [0.263, 0.281]	0.897	0.0050
	i→t	0.116	0.078	0.461 [0.440, 0.481]	0.204	0.196	0.692	0.489	**0.413**	0.380	0.286 [0.264, 0.309]	0.866	0.0049
	t→i	0.151	0.098	0.475 [0.456, 0.494]	0.300	0.293	0.702	0.486	**0.438**	0.375	0.331 [0.314, 0.349]	0.867	0.0035
	t→t	0.109	0.073	0.461 [0.445, 0.477]	0.221	0.218	0.700	0.470	**0.500**	0.336	0.274 [0.261, 0.287]	0.865	0.0045
Qwen3-VL	i→i	0.085	0.054	0.481 [0.472, 0.490]	0.296	0.145	0.693	0.491	0.235	**0.408**	0.233 [0.225, 0.240]	0.858	0.0042
	i→t	0.102	0.076	0.364 [0.339, 0.393]	0.104	0.072	0.621	0.393	**0.295**	0.270	0.203 [0.181, 0.226]	0.832	0.0056
	t→i	0.189	0.146	0.317 [0.279, 0.351]	0.109	0.083	0.565	0.325	**0.278**	0.238	0.236 [0.216, 0.253]	0.812	0.0038
	t→t	0.106	0.069	0.505 [0.481, 0.534]	0.157	0.146	0.708	0.513	**0.564**	0.377	0.314 [0.289, 0.341]	0.880	0.0034

Table 10: Accuracy of the synopsis's two estimates on the held-out queries ($K = 10{,}000$), averaged over non-flat configurations: estimated fidelity against the global average (GA) and the density-only estimate (DO), and estimated coverage. ECE: pooled over the pairs of query and configuration of all nine configurations, as in Figure 4. Bold: the better-predicted of interior and boundary queries. Terms: Table 8.

and it separates queries that meet the target from those that do not well (AUC above 0.8 everywhere). It is also well calibrated (Figure 4): among queries for which the synopsis predicts a coverage of, say, 0.8, close to 80% meet the target. Calibration is what makes the reliability level meaningful: if the coverage is calibrated, a selector that controls the expected number of misses controls the actual number.

Location predicts better for uni-modal search. For SigLIP2, OpenCLIP, and Qwen3-VL, skill is higher for both uni-modal directions

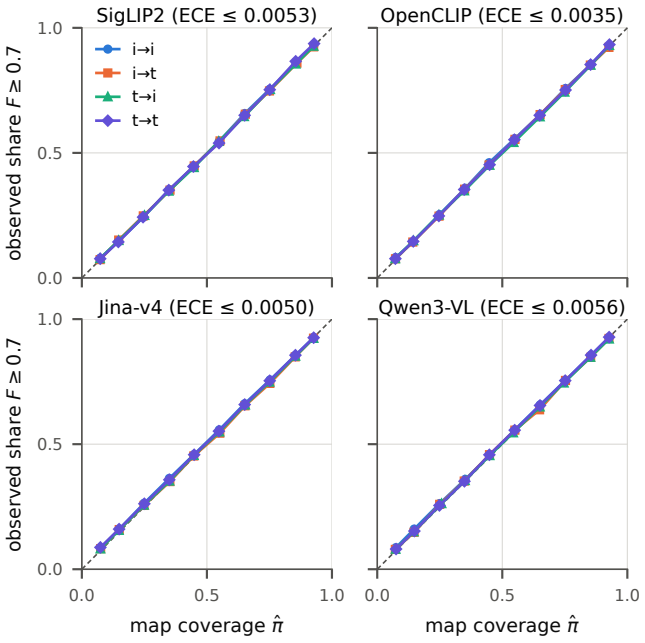


Figure 4: Calibration of the estimated coverage ($\tau = 0.7$, $K = 10{,}000$, nine configurations pooled): mean predicted coverage against the share of held-out (query, configuration) pairs that meet the target, in 10 bins over pairs with $0.05 < \hat{\pi} < 0.95$. Dashed: perfect calibration.

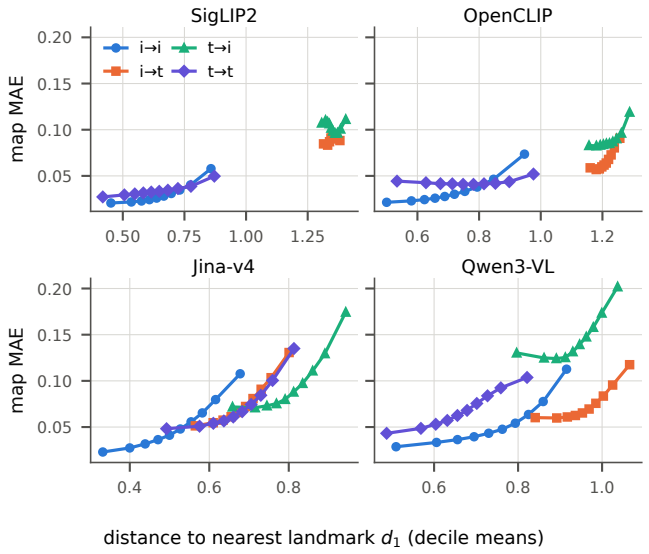


Figure 5: Error of the estimated fidelity (mean absolute error over non-flat configurations) against the distance d_1 from a held-out query to its nearest landmark, in ten equal-count bins per direction ($K = 10{,}000$).

	HNSW		IVFPQ		O-IVFPQ	
Model	Shared	Keyed [CI of diff.]	Shared	Keyed [CI]	Shared	Keyed [CI]
SigLIP2	0.256	**0.466 [0.205, 0.216]**	0.283	**0.445 [0.128, 0.244]**	0.278	**0.442 [0.129, 0.247]**
OpenCLIP	0.261	**0.516 [0.244, 0.268]**	0.167	**0.451 [0.211, 0.471]**	0.162	**0.450 [0.215, 0.474]**
Jina-v4	0.252	**0.430 [0.173, 0.184]**	0.434	**0.506 [0.070, 0.074]**	0.435	**0.506 [0.069, 0.074]**
Qwen3-VL	−0.021	**0.312 [0.317, 0.352]**	0.324	**0.469 [0.141, 0.149]**	0.324	**0.469 [0.141, 0.150]**

Table 11: Skill of the keyed synopsis (statistics per configuration, bold) and the shared synopsis (one offset per cell), per index family, averaged over directions and non-flat configurations ($K = 10{,}000$). Brackets: 95% confidence interval of the difference.

than for both cross-modal ones, as the smoothness premise predicts (for Jina-v4, cross-modal skill, 0.461 and 0.475, is as high as its text-to-text skill, 0.461): a cross-modal query is located among the landmarks of the other modality, which represent its surroundings less well. Measured by the mean distance from a query to its nearest landmark, cross-modal queries lie about twice as far as uni-modal ones for SigLIP2, and less far for OpenCLIP, Qwen3-VL, and Jina-v4. Jina-v4, with the smallest gap, on average, has the highest cross-modal skill, but Qwen3-VL, with a smaller gap than SigLIP2 and OpenCLIP, has cross-modal skill as low as theirs, so the gap explains the difference only in part.

Queries near cell boundaries. The premise is least plausible near the boundary between two cells. We compare *interior* queries, the third whose nearest landmark is much closer than the second-nearest, with *boundary* queries, the third for which the two are almost equally close (bold in Table 10: the better-predicted group). For text-to-text search, interior queries are predicted much better in all four models; for cross-modal search, better but by less. For image-to-image search, skill is higher for boundary queries under OpenCLIP, Jina-v4 and Qwen3-VL. This is an artifact of how skill is normalized: skill in each group is measured against that group's own variance, and interior image queries vary much less. In absolute error, interior queries are predicted more accurately in all 16 settings, as the premise expects.

Figure 5 shows the same continuously: in every setting, the synopsis's error is higher for the queries farthest from their nearest landmark than for the nearest ones. The cross-modal queries of SigLIP2 and OpenCLIP lie far from all landmarks, and their error is high throughout.

Answer to RQ2. Yes. A query's location predicts its fidelity in every setting, the synopsis captures almost all of the variation between cells available at its resolution, and its coverage estimate is well calibrated. Location is more informative for uni-modal than for cross-modal search, and less informative near cell boundaries, as the smoothness premise predicts.

5.3 RQ3: Must the synopsis be kept per configuration?

The configuration-specificity premise says that the same region can be easy for one configuration and hard for another, so statistics must be kept for every pair of cell and configuration. We compare the synopsis with a *shared* variant that keeps one difficulty value per cell: the average, over configurations, of how far the cell's mean fidelity lies from the configuration's global mean. The shared synopsis knows which regions are harder or easier than average,

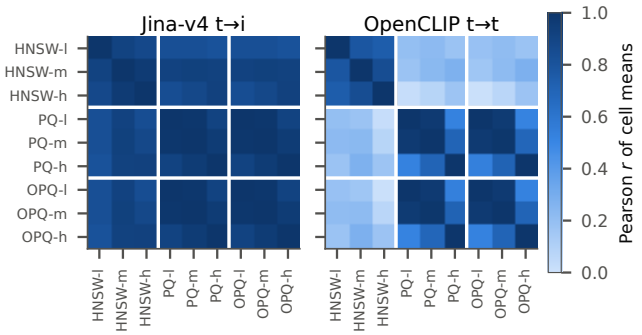


Figure 6: Correlation, across cells, of the mean fidelity of every pair of configurations (K = 10,000; l/m/h: low/moderate/high tier), in the setting where index families agree most (left, Jina-v4 t→i) and least (right, OpenCLIP t→t).

but assumes they are equally so for every configuration. Table 11 reports both per index family.

Keeping the synopsis per configuration is always better. The keyed synopsis has higher skill than the shared synopsis for every model and index family, with confidence intervals for the difference that exclude zero and every per-configuration test significant. The shared synopsis is not useless, since its skill is positive in all but one combination: some regions are harder than others for every configuration alike. But one value per cell captures only part of what the keyed synopsis captures.

The gap follows the index families. The shared offset averages over all nine configurations, six of which are IVF-family, so it mostly describes how IVF-family configurations behave. For Jina-v4 and Qwen3-VL, the two synopses are therefore close for the IVF families but far apart for HNSW, whose pattern of easy and hard regions differs. For SigLIP2 and OpenCLIP, the gap is large for the IVF families as well: there, HNSW and IVF-family configurations agree much less on which regions are hard (Figure 6), so no single offset fits both. Configurations of the same family largely agree with each other. Giving each configuration its own scale on the shared offset narrows the gap but does not close it in any model or family: the regional pattern differs across configurations, not only its size.

Configurations differ in how much fidelity they lose, not mainly in their order. One might expect configuration specificity to show up as reversals: regions where configuration A beats B and others where B beats A. Table 12 shows that reversals are the exception: in most settings, no pair of the 36 formed by the nine configurations reverses in a substantial share of cells, and the pair that reverses most often is usually lopsided. Two-way reversals occur only in four cross-modal settings (bold). In a given region, configurations largely agree on which of them is better but differ in how much fidelity they lose, and the latter is what selection depends on: whether a configuration meets the target in a region changes with the size of its loss even when the order of configurations does not.

Answer to RQ3. Yes. A single difficulty value per cell carries some information, but keeping statistics per configuration improves skill

Model	Dir.	Pair A / B (Similar cost)	A wins	B wins	#pairs
SigLIP2	i→i	hnsw_high / ivfpq_mod	2.5%	0.3%	0
	i→t	hnsw_high / oivfpq_high	3.7%	17.6%	0
	t→i	hnsw_high / oivfpq_high	13.7%	1.2%	**2**
	t→t	hnsw_high / oivfpq_mod	0.0%	4.9%	0
OpenCLIP	i→i	hnsw_high / ivfpq_mod	1.3%	0.2%	0
	i→t	hnsw_high / oivfpq_mod	13.1%	9.1%	**2**
	t→i	hnsw_high / oivfpq_high	9.0%	2.0%	**4**
	t→t	hnsw_high / oivfpq_high	0.0%	64.2%	0
Jina-v4	i→i	hnsw_high / oivfpq_high	0.3%	0.2%	0
	i→t	oivfpq_mod / hnsw_mod	0.6%	20.4%	0
	t→i	ivfpq_mod / hnsw_mod	0.7%	15.7%	0
	t→t	oivfpq_mod / hnsw_mod	1.0%	10.8%	0
Qwen3-VL	i→i	hnsw_high / ivfpq_high	0.1%	0.3%	0
	i→t	oivfpq_mod / hnsw_mod	0.8%	11.4%	0
	t→i	hnsw_high / oivfpq_high	16.3%	12.4%	**4**
	t→t	oivfpq_mod / hnsw_mod	0.6%	21.6%	0

Table 12: Rank reversals between configurations of different families with similar cost (within 2×; A the cheaper), over cells with at least $q_{\min}$ sample queries (K = 10,000). For each setting: the pair that reverses most often, the share of cells in which each wins by more than 0.1 in mean fidelity, and how many of the 36 pairs reverse in at least 5% of cells each way (bold: any).

for every model and index family, most where index families behave differently. Configurations largely agree on which of them is better in a region and disagree on how much fidelity the region costs them, which is what selection depends on. Keeping per-configuration statistics costs little: the synopsis is a small fraction of the size of a single index (Section 5.5).

5.4 RQ4: Does synopsis-guided selection reduce cost without missing the target?

Table 13 compares the static plan with four ways of using the synopsis at $\tau = 0.7$: the mean rule, the coverage rule, and the budget rule at the static rate and at the requirement. Figure 7 shows, for each setting, the full trade-off between cost and violation rate as the coverage rule's ρ or the budget rule's λ varies.

Under the requirement, the synopsis lowers cost in every setting. Held to the requirement, at most 10% of queries missing the target, the budget rule is cheaper than the static plan in all 16 settings, at a median of 0.72× its cost, and recovers more than half of the saving that the optimum at the requirement achieves in a typical setting. In one setting, SigLIP2 i→t, no single configuration meets the requirement on Q, and the static plan violates it on 10.1% of held-out queries. There, the budget rule is both cheaper and within the requirement, and in the other 15 settings, both meet it. Its violation rate on the held-out queries stays within the requirement in every setting; in half of them, the upper end of its confidence interval exceeds 10%, by at most a little over one percentage point. The comparison is fair because both selectors face the same requirement: the static plan misses less often than it is allowed to only because a catalog offers few configurations, and the synopsis spends that unused allowance on cheaper configurations where its estimates say they are safe. However, this saving is not merely the allowance spent blindly, as the comparison at the static plan's own violation

		Static plan		Synopsis, mean rule			Synopsis, coverage rule (ρ=0.9)			Synopsis, budget (static viol.)			Synopsis, budget ($\leq 1-\rho$)			Opt. cost	
Model	Dir.	Viol.	Short.	Cost	Viol.	Rec.	Cost	Viol.	Rec.	Cost	Viol.	Rec.	Cost	Viol. [95% CI]	Rec.	matched	$\leq 1-\rho$
SigLIP2	i→i	0.8%	0.11	0.72× [0.71, 0.72]	8.0%	89%	1.03× [1.00, 1.05]	1.5%	−8%	1.00× [0.99, 1.02]	0.8%	−2%	**0.69× [0.68, 0.69]**	9.6% [9.3, 9.9]	82%	0.68×	0.62×
	i→t	*10.1%*[§]	0.14	0.30× [0.24, 0.35]	*19.5%*	102%	0.62× [0.56, 0.67]	8.6%	55%	0.54× [0.47, 0.60]	9.9%	67%	**0.54× [0.47, 0.60]**	9.9% [8.5, 11.3]	59%	0.31×	0.21×
	t→i	2.6%	0.11	0.18× [0.17, 0.19]	9.9%	103%	0.42× [0.38, 0.46]	4.0%[†]	73%	0.73× [0.71, 0.75]	2.6%	34%	**0.14× [0.14, 0.14]**	9.9% [9.4, 10.4]	96%	0.20×	0.11×
	t→t	3.4%	0.15	0.96× [0.95, 0.99]	5.7%	29%	1.17× [1.15, 1.20]	1.8%	−138%	0.99× [0.99, 1.00]	3.1%	6%	**0.88× [0.87, 0.89]**	9.7% [9.3, 10.1]	65%	0.87×	0.82×
OpenCLIP	i→i	2.5%	0.12	0.85× [0.84, 0.87]	9.0%	59%	1.54× [1.50, 1.58]	2.1%	−218%	1.02× [1.01, 1.03]	2.5%	−9%	**0.79× [0.78, 0.79]**	9.0% [8.7, 9.2]	73%	0.75×	0.71×
	i→t	2.8%	0.13	0.35× [0.33, 0.36]	*19.6%*	98%	0.74× [0.73, 0.75]	5.2%	39%	0.96× [0.95, 0.96][‡]	2.9%	7%	**0.53× [0.52, 0.55]**	9.2% [8.8, 9.6]	62%	0.33×	0.24×
	t→i	1.6%	0.09	0.17× [0.16, 0.18]	*11.5%*	104%	0.40× [0.36, 0.45]	3.4%	75%	0.70× [0.67, 0.74]	1.6%	37%	**0.13× [0.13, 0.14]**	9.9% [9.4, 10.5]	96%	0.20×	0.10×
	t→t	0.9%	0.11	0.44× [0.43, 0.45]	9.2%	98%	0.62× [0.61, 0.63]	2.2%	66%	0.82× [0.81, 0.83]	0.9%	31%	**0.42× [0.41, 0.42]**	9.3% [9.0, 9.7]	89%	0.43×	0.34×
Jina-v4	i→i	9.1%	0.13	0.72× [0.72, 0.73]	*16.8%*	72%	1.03× [1.02, 1.04]	4.9%[†]	−8%	0.80× [0.79, 0.80]	9.1%	53%	**0.78× [0.78, 0.79]**	9.8% [9.6, 10.1]	50%	0.62×	0.57×
	i→t	2.8%	0.13	0.31× [0.30, 0.33]	*25.0%*	98%	0.52× [0.50, 0.53]	6.6%[†]	69%	0.81× [0.80, 0.83]	2.8%	27%	**0.39× [0.39, 0.40]**	9.1% [8.5, 9.9]	78%	0.30×	0.22×
	t→i	6.3%	0.16	0.43× [0.42, 0.44]	*26.0%*	94%	0.67× [0.66, 0.68]	9.8%[†]	54%	1.04× [1.03, 1.05][‡]	6.5%	−6%	**0.62× [0.61, 0.63]**	9.7% [9.1, 10.2]	55%	0.39×	0.31×
	t→t	8.4%	0.20	0.76× [0.75, 0.76]	*22.3%*	63%	1.20× [1.18, 1.22]	7.0%[†]	−52%	0.95× [0.94, 0.95]	8.2%	14%	**0.87× [0.87, 0.88]**	9.8% [9.5, 10.1]	29%	0.61×	0.56×
Qwen3-VL	i→i	9.9%	0.13	0.77× [0.76, 0.78]	*17.7%*	59%	1.24× [1.22, 1.26]	4.7%[†]	−62%	0.90× [0.89, 0.90]	9.1%	27%	**0.90× [0.89, 0.90]**	9.1% [8.9, 9.3]	23%	0.61×	0.55×
	i→t	9.4%	0.15	0.76× [0.71, 0.84]	*21.5%*	59%	1.48× [1.41, 1.56]	5.8%[†]	−118%	0.92× [0.90, 0.93]	9.2%	21%	**0.89× [0.88, 0.91]**	9.6% [8.9, 10.6]	22%	0.60×	0.52×
	t→i	7.4%	0.30	0.48× [0.43, 0.53]	*22.6%*	83%	1.03× [0.99, 1.10]	5.0%	−5%	0.86× [0.84, 0.88]	7.2%	22%	**0.78× [0.75, 0.81]**	8.7% [8.2, 9.3]	30%	0.37×	0.27×
	t→t	6.4%	0.17	0.69× [0.68, 0.71]	*18.7%*	67%	1.21× [1.19, 1.24]	5.3%[†]	−46%	0.90× [0.89, 0.92]	6.1%	21%	**0.75× [0.74, 0.75]**	9.6% [9.4, 9.9]	48%	0.54×	0.47×

Table 13: Plan selection at $\tau = 0.7$ ($K = 10{,}000$). Costs are relative to the static plan, include the lookup, and carry 95% confidence intervals. The optimum needs no lookup. Bold: our method, the budget rule at the requirement; grey: not cheaper than the static plan; italic: violation rate above the requirement of 10%. Terms and markers: Table 8.

rate below shows. Figure 7 places each selector on the full trade-off curve, so that the comparison can also be read at any other violation rate. Every point on a curve corresponds to a different reliability level. An operator who needs $\rho = 0.95$, for instance, reads the cost at a 5% violation rate. Our results are therefore not tied to $\rho = 0.9$.

At the static plan's own violation rate, it still saves in most settings. Held to the lower violation rate the static plan actually incurs, the budget rule has less room. Where that rate is reachable on Q (14 settings), it meets it on the held-out queries and is cheaper than the static plan in 12, by at least 8% in 10. It recovers a smaller share of the matched optimum's saving, about a fifth in a typical setting. This comparison also separates the synopsis's information from the allowance. At the static plan's own violation rate, a deployment that ignores where a query lies has no allowance left to spend. Every configuration cheaper than the static plan violates the target on more than 10% of queries. Sending a share of queries to one of them at random would therefore raise the violation rate above the static plan's. The saving at this rate comes from the synopsis's estimates, not from the allowance. A random blend of a cheaper and a costlier configuration could in principle match the static plan's violation rate at lower cost, if the costlier configuration is accurate enough. Such a blend uses no information about the query either. Chosen on Q to match the static plan's violation rate, it is cheaper than the static plan in only one setting (Jina-v4 i→i, 0.95×).

The simpler rules fall short. The mean rule is cheap only because it misses often: a cell mean at the target leaves many of the cell's queries below it, and the rule misses the target on more than 10% of queries in most settings (italic). The coverage rule controls misses, as its calibration predicts, but not cost: requiring coverage ρ in every cell rather than over the workload moves cells where the static plan's configuration falls short to costlier ones, and it costs more than the static plan in nine settings (grey). Against the mean rule, its per-query shortfall is significantly lower in every setting (Wilcoxon–Pratt, Benjamini–Hochberg $q < 0.05$), mostly because it misses far fewer queries; the queries it does miss fall less far below the target in half of the settings (†). The budget rule's curve in Figure 7 lies on or below the coverage rule's in most settings, so for a given violation budget it is the better way to use the synopsis. All rules differ significantly from the static plan in which queries they miss (McNemar), in all but three of the 64 comparisons.

Where the synopsis gains little. At the static rate, the synopsis saves nothing for SigLIP2 and OpenCLIP i→i, although the optimum would save a quarter to a third: the only cheaper configuration, `hnsw_low`, serves part of the queries, and the synopsis cannot single out which ones reliably enough. More generally, half of the variation in fidelity lies within cells (RQ2), where the synopsis cannot see it, which is why the budget rule captures only part of the optimum's saving.

Other targets. Table 14 repeats the evaluation at $\tau = 0.8$ and 0.9. A higher target leaves fewer feasible settings. Even the optimum can meet the requirement in only 15 settings at $\tau = 0.8$ and 9 at $\tau = 0.9$ (infeas.). In the others, more than $1 - \rho$ of the queries miss the target under every configuration. The budget rule reaches the requirement on Q in 14 and 6 settings. It cannot reach it where the optimum cannot, and in four more settings: Jina-v4 t→i at $\tau = 0.8$, and OpenCLIP t→i and Qwen3-VL i→t and t→t at $\tau = 0.9$. There, the optimum is feasible, but the synopsis's estimates are not precise enough to reach $1 - \rho$ (‡).

Where the budget rule reaches the requirement, its violation rate on the held-out queries ranges from 9.3 to 9.9% at $\tau = 0.8$ and from 8.5 to 9.7% at $\tau = 0.9$. At $\tau = 0.8$, it is cheaper than the static plan in 13 of 14 settings. In the remaining one, SigLIP2 t→t, it matches the static plan (0.999×) and recovers almost none of the optimum's saving. At $\tau = 0.9$, it is cheaper in all 6 settings. Its median cost is 0.73× at $\tau = 0.8$, close to the 0.72× at $\tau = 0.7$, and 0.45× at $\tau = 0.9$. It recovers a median of 56% and 72% of the saving of the optimum at the requirement, which itself costs a median of 0.44× and 0.21× the static plan. The six feasible settings at $\tau = 0.9$ are all uni-modal, so medians are not directly comparable across targets.

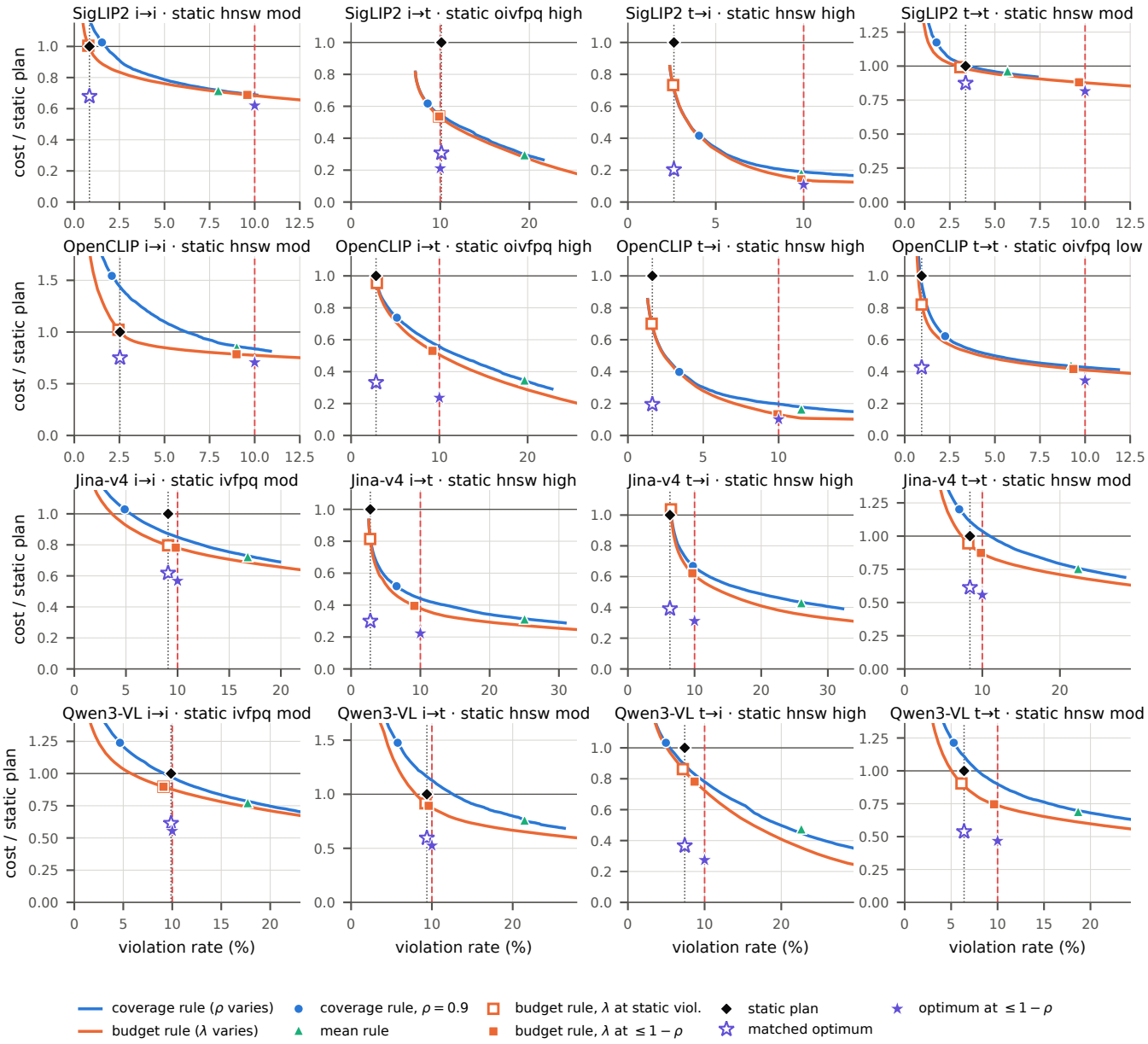


Figure 7: Cost against violation rate on the held-out queries ($\tau = 0.7$, $K = 10{,}000$) as ρ (coverage rule) or λ (budget rule) varies. Markers: static plan, mean rule, coverage rule at $\rho = 0.9$, budget rule at the static rate (open square) and at the requirement (filled square), matched optimum (open star), and optimum at the requirement (filled star). Dotted: the static plan's violation rate; dashed: the requirement.

A higher target moves the static plan to costlier configurations. `hnsw_high` is the static plan in 5 settings at $\tau = 0.7$, 8 at $\tau = 0.8$, and 10 at $\tau = 0.9$, and a high IVF-family tier in 2, 2, and 5. Most queries do not need these configurations: at $\tau = 0.9$, the optimum costs as little as 0.12× the static plan for SigLIP2 t⟶t and OpenCLIP i⟶i. The saving therefore does not depend on our default target.

Answer to RQ4. Yes, under the requirement a static deployment is built to meet: synopsis-guided selection lowers cost in every setting, to a median of 0.72× the static plan, lookup included. Even at the static plan's lower realized violation rate, it lowers cost in most settings. The simpler rules show why the budget rule is needed: the cell mean ignores how fidelity is spread within a cell, and per-cell coverage overpays in hard cells.

5.4.1 Does the saving need the full catalog? Per-query selection requires every configuration it may choose to be built and kept in memory. Our nine-configuration catalog occupies 140–376 GB

		$\tau = 0.8$						$\tau = 0.9$					
		Static plan		Budget, $\leq 1-\rho$			Opt.	Static plan		Budget, $\leq 1-\rho$			Opt.
Model	Dir.	Config.	Viol.	Cost [95% CI]	Viol. [95% CI]	Rec.	$\leq 1-\rho$	Config.	Viol.	Cost [95% CI]	Viol. [95% CI]	Rec.	$\leq 1-\rho$
SigLIP2	i→i	hnsw_mod	2.4%	**0.83× [0.82, 0.84]**	9.5% [9.2, 9.8]	68%	0.75×	hnsw_mod	8.5%	**0.98× [0.98, 0.99]**	9.6% [9.2, 10.0]	14%	0.88×
	i→t	oivfpq_high*	*21.5%*	0.88× [0.86, 0.90]‡	*17.8% [15.1, 20.4]*	–	infeas.	oivfpq_high*	*45.4%*	0.93× [0.92, 0.94]‡	*42.4% [38.7, 46.1]*	–	infeas.
	t→i	hnsw_high	6.6%	**0.44× [0.41, 0.48]**	9.6% [9.1, 10.0]	69%	0.20×	hnsw_high*	*19.8%*	1.04× [1.04, 1.05]‡	*17.9% [17.0, 18.8]*	–	infeas.
	t→t	hnsw_mod	9.5%	1.00× [1.00, 1.00]	9.6% [9.2, 10.0]	1%	0.88×	oivfpq_high	0.6%	**0.32× [0.31, 0.32]**	9.7% [9.4, 9.9]	78%	0.12×
OpenCLIP	i→i	hnsw_mod	5.7%	**0.91× [0.91, 0.91]**	9.5% [9.2, 9.8]	45%	0.80×	hnsw_high	5.7%	**0.18× [0.18, 0.19]**	8.9% [8.6, 9.2]	92%	0.12×
	i→t	oivfpq_high	6.7%	**0.85× [0.84, 0.86]**	9.9% [9.2, 10.6]	32%	0.52×	oivfpq_high*	*17.9%*	1.00× [1.00, 1.00]‡	*18.0% [16.8, 19.2]*	–	infeas.
	t→i	hnsw_high	5.3%	**0.40× [0.36, 0.44]**	9.5% [8.4, 11.0]	76%	0.20×	hnsw_high*	*17.7%*	1.09× [1.08, 1.10]‡	*13.3% [12.6, 14.0]*	−15%	0.44×
	t→t	oivfpq_low	8.4%	**0.62× [0.61, 0.63]**	9.6% [9.3, 9.9]	72%	0.47×	oivfpq_high	0.4%	**0.57× [0.56, 0.58]**	8.5% [8.1, 8.7]	55%	0.22×
Jina-v4	i→i	hnsw_mod	8.3%	**0.94× [0.94, 0.95]**	9.3% [9.1, 9.6]	14%	0.60×	hnsw_high	5.2%	**0.56× [0.55, 0.56]**	9.7% [9.4, 10.0]	66%	0.33×
	i→t	hnsw_high	6.7%	**0.64× [0.62, 0.65]**	9.4% [8.9, 10.1]	53%	0.32×	hnsw_high*	*17.2%*	1.07× [1.07, 1.08]‡	*16.2% [15.2, 17.3]*	–	infeas.
	t→i	hnsw_high*	*12.1%*	1.12× [1.11, 1.13]‡	*11.9% [11.0, 12.8]*	−21%	0.42×	hnsw_high*	*25.3%*	1.21× [1.21, 1.22]‡	*23.5% [22.4, 24.7]*	–	infeas.
	t→t	hnsw_high	6.8%	**0.58× [0.57, 0.59]**	9.7% [9.5, 10.0]	59%	0.29×	hnsw_high*	*17.7%*	1.12× [1.12, 1.13]‡	*16.3% [15.8, 16.8]*	–	infeas.
Qwen3-VL	i→i	hnsw_mod	6.0%	**0.83× [0.83, 0.83]**	9.9% [9.6, 10.2]	35%	0.52×	hnsw_high	3.0%	**0.34× [0.34, 0.35]**	9.6% [9.3, 9.8]	83%	0.21×
	i→t	hnsw_high	4.9%	**0.54× [0.52, 0.56]**	9.6% [9.3, 10.0]	61%	0.24×	ivfpq_high*	*11.5%*	0.82× [0.81, 0.83]‡	*11.1% [10.3, 12.3]*	24%	0.23×
	t→i	hnsw_high*	*11.4%*	**0.97× [0.96, 0.98]**	9.5% [8.9, 10.1]	5%	0.40×	hnsw_high*	*26.2%*	1.26× [1.22, 1.30]‡	*20.0% [19.2, 20.8]*	–	infeas.
	t→t	hnsw_high	3.3%	**0.33× [0.32, 0.34]**	9.3% [9.1, 9.6]	80%	0.17×	hnsw_high*	*12.0%*	1.18× [1.17, 1.19]‡	*10.8% [10.4, 11.1]*	−27%	0.31×
Median (feasible)				**0.73×**		56%	0.44×			**0.45×**		72%	0.21×

Table 14: Plan selection at $\tau = 0.8$ and 0.9 ($K = 10{,}000$, held-out queries), in the format of Table 13. Costs are relative to the static plan at the same target and include the lookup. Bold: our method. Grey: not cheaper than the static plan. Italic: violation rate above the requirement of 10%. *: no configuration meets the target on a fraction ρ of Q, so the static plan is the one that meets it most often. ‡: the budget rule cannot reach $1-\rho$ on Q, so the λ with the fewest violations is used. Rec.: the share of the saving of the optimum at the requirement that the budget rule achieves. infeas.: more than $1-\rho$ of the queries cannot be served by any configuration. Medians are over the feasible settings. Terms: Table 8.

		Full catalog (nine indexes)					HNSW-only (three indexes)					Single index (three efSearch)					Union (eleven configurations)				
				Budget, $\leq 1-\rho$		Budget			Budget, $\leq 1-\rho$		Budget			Budget, $\leq 1-\rho$		Budget			Budget, $\leq 1-\rho$		Budget
Model	Dir.	Static	Opt.	Cost	Viol.	static v.	Static	Opt.	Cost	Viol.	static v.	Static	Opt.	Cost	Viol.	static v.	Static	Opt.	Cost	Viol.	/ static
SigLIP2	i→i	H-m	0.62×	0.69×	9.6%	1.00×	H-m	0.62×	0.69×	9.6%	1.01×	l	0.21×	0.22×	6.9%	0.22×	S-l	0.21×	0.22×	6.9%	1.07×
	i→t	O-h*	0.21×	0.54×	9.9%	0.54×	H-h*	infeas.	0.65‡×	12.1%	0.65‡×	h*	infeas.	0.68‡×	12.1%	0.68‡×	O-h*	0.18×	0.54×	9.9%	**0.54×**
	t→i	H-h	0.11×	0.14×	9.9%	0.73×	H-h	0.11×	0.15×	9.5%	0.81‡×	h	0.16×	0.38×	9.9%	0.98‡×	H-h	0.04×	0.15×	9.4%	**0.15×**
	t→t	H-m	0.82×	0.88×	9.7%	0.99×	H-m	0.82×	0.88×	9.7%	1.01×	h	0.28×	0.62×	9.5%	6.55‡×	H-m	0.22×	0.35×	9.8%	**0.35×**
OpenCLIP	i→i	H-m	0.71×	0.79×	9.0%	1.02×	H-m	0.71×	0.79×	9.0%	1.04×	l	0.32×	0.34×	7.9%	0.34×	S-l	0.32×	0.34×	7.9%	1.08×
	i→t	O-h	0.24×	0.53×	9.2%	0.96‡×	H-h*	infeas.	0.46‡×	25.3%	0.19×	h*	infeas.	0.61‡×	30.8%	0.61‡×	O-h	0.20×	0.52×	9.4%	**0.52×**
	t→i	H-h	0.10×	0.13×	9.9%	0.70×	H-h	0.10×	0.15×	9.3%	0.80‡×	m	0.05×	0.13×	9.6%	0.14×	S-m	0.05×	0.13×	9.5%	1.00×
	t→t	O-l	0.34×	0.42×	9.3%	0.82×	H-m*	0.39×	1.08‡×	10.7%	0.40×	h*	infeas.	2.95‡×	20.3%	2.95‡×	O-l	0.18×	0.32×	9.7%	**0.32×**
Jina-v4	i→i	I-m	0.57×	0.78×	9.8%	0.80×	H-m	0.63×	0.81×	9.9%	1.07×	m	0.44×	0.60×	9.6%	0.71×	S-m	0.44×	0.59×	9.7%	**0.80×**
	i→t	H-h	0.22×	0.39×	9.1%	0.81×	H-h	0.28×	0.41×	9.9%	0.90‡×	h	0.25×	0.46×	9.1%	0.98‡×	H-h	0.19×	0.39×	8.9%	**0.39×**
	t→i	H-h	0.31×	0.62×	9.7%	1.04‡×	H-h	0.38×	0.64×	9.8%	0.96‡×	h	0.34×	0.71×	9.9%	1.01‡×	H-h	0.26×	0.63×	9.5%	**0.63×**
	t→t	H-m	0.56×	0.87×	9.8%	0.95×	H-m	0.70×	1.00×	9.6%	1.05×	h	0.51×	0.96×	9.5%	2.33‡×	H-m	0.42×	0.83×	9.4%	**0.83×**
Qwen3-VL	i→i	I-m	0.55×	0.90×	9.1%	0.90×	H-m	0.72×	1.03×	9.0%	1.39×	m	0.58×	0.70×	9.6%	0.99×	I-m	0.47×	0.70×	9.6%	**0.70×**
	i→t	H-m	0.52×	0.89×	9.6%	0.92×	H-m	0.81×	1.04×	9.7%	1.06×	h	0.71×	1.48×	9.0%	3.23‡×	H-m	0.42×	0.89×	9.6%	**0.89×**
	t→i	H-h	0.27×	0.78×	8.7%	0.86×	H-h	0.35×	0.82×	9.9%	1.00‡×	h	0.41×	0.86×	9.7%	1.00‡×	H-h	0.24×	0.78×	8.9%	**0.78×**
	t→t	H-m	0.47×	0.75×	9.6%	0.90×	H-m	0.73×	0.95×	9.7%	1.05×	h	0.48×	0.83×	9.5%	3.25‡×	H-m	0.38×	0.70×	9.5%	**0.70×**

Table 15: Plan selection with smaller catalogs and their union at $\tau = 0.7$. All costs are relative to the static plan of the nine-configuration catalog, except the last column, which is relative to the union's own static plan (bold: cheaper; grey: not cheaper). Abbreviations and markers: Table 8.

per model, most of it in the three HNSW indexes (Table 16). We ask how much of the saving survives with less, using two smaller catalogs and one larger one:

- *HNSW-only*: the three HNSW configurations, still three separately built indexes.
- *Single index*: the index of `hnsw_high`, searched with three values of `efSearch`. The highest is `hnsw_high`'s own; the two lower ones are, per model, the first values of the parameter sweep at which mean fidelity over the four directions reaches 0.7 and 0.8 (`efSearch` of 192 and 384 for SigLIP2, 256 and 1,024 for OpenCLIP, 48 and 96 for Jina-v4, and 64 and 128 for Qwen3-VL). All three share one index, so

	Full	HNSW	Single	Union
Indexes per collection	9	3	1	9
Memory (GB), SigLIP2	208.7	163.9	55.2	208.7
Memory (GB), OpenCLIP	140.4	110.4	37.4	140.4
Memory (GB), Jina-v4	375.8	288.7	96.8	375.8
Memory (GB), Qwen3-VL	375.8	288.7	96.8	375.8
Labeling Q (CPU-h)	344–5,036	94–767	84–705	369–5,086
Opt., median	0.41×	0.63×$^{(14)}$	0.34×$^{(13)}$	0.23×
Budget, median	0.72×	0.81×$^{(13)}$	0.62×$^{(13)}$	0.53×
Budget / own static, med.	0.72×	0.78×$^{(13)}$	0.68×$^{(13)}$	0.70×

Table 16: The catalogs of Section 5.4.1. Memory: image and text indexes together; the union adds no index. Costs: medians over the settings where the catalog can meet the requirement (their number in parentheses where fewer than 16), relative to the nine-configuration static plan, except the last row, which is relative to each catalog's own static plan.

	SigLIP2	OpenCLIP	Jina-v4	Qwen3-VL
Dimension d	1152	768	2048	2048
c_{est}, 1 query, mean (μs)	1640	822	*2859*	*2842*
c_{est}, 1 query, p99 (μs)	1659	848	2874	2873
Cheapest plan, 1 query (μs)	1678	1482	445	349
c_{est}, batched (μs/q)	**23.3**	**20.6**	**31.4**	**32.3**
Cheapest plan, batched (μs/q)	665	490	223	163
Synopsis size (MiB)	119.5	90.2	187.8	187.8
k-means (GPU-h)	0.12	0.08	0.21	0.20
Exact k-NN on Q (GPU-h)	0.57	0.40	0.98	0.98
All configs on Q (h, 64 thr.)	78.7	51.2	5.4	8.8
All configs on Q (CPU-h)	5,036	3,275	344	563

Table 17: Cost of the synopsis at K = 10,000, averaged over directions: the lookup, one query at a time and batched, against the cheapest configuration (`hnsw_low`); the synopsis's size; and the one-time offline cost. Bold: the batched regime used throughout; italic: single-query lookup costlier than the cheapest configuration.

per-query selection adds no memory. Their costs were measured on one node together with `hnsw_high` and put on the scale of the main benchmark through it.

- *Union*: the nine configurations plus the single index's two lower settings, eleven configurations with no additional index. Its static plan is the best static choice among everything we built, the strongest static baseline available without new indexes.

For each catalog, the static plan, the optimum, and the synopsis are computed within that catalog alone, so that a static and a per-query deployment choose among the same configurations. The synopsis uses the same landmarks; only the configurations it records change. Table 15 reports every cost relative to the static plan of the nine-configuration catalog, so that all catalogs share one scale.

Against the strongest static baseline, the synopsis still saves. The union's static plan is the nine-configuration static plan in 12 of 16 settings: the cheaper single-index settings do not help a static deployment there, because its hardest queries still need the same configuration. In the other four, the union's static plan is a cheap single-index setting, which shows that `hnsw_low` is a poor operating point for these models. Measured against the union's own static plan, the budget rule at the requirement costs a median of 0.70×, about the same as with the nine-configuration catalog. It is cheaper in 13 settings, equal in one, and slightly costlier, by about its lookup cost, in the two where the union's static plan is already its cheapest setting (grey in Table 15).

A smaller catalog keeps most of the opportunity. Without the IVF-family indexes, the optimum keeps most of the full catalog's saving in a typical setting. The loss lies in Jina-v4 and Qwen3-VL, whose low and moderate IVF tiers are cheaper than `hnsw_mod`, and in SigLIP2 and OpenCLIP i⟶t, where only IVF-family configurations meet the target often enough to stay within the requirement. The single index differs from the full catalog in both directions: its lower settings are cheaper and more accurate than `hnsw_low` for SigLIP2, OpenCLIP, and Jina-v4, but its single grid of `efSearch` values per model leaves the lower settings short of the target in some directions, where the only remaining choice is `hnsw_high`'s expensive setting, and in three settings no assignment meets the requirement.

The synopsis captures it within each catalog. Within each smaller catalog, the budget rule at the requirement is cheaper than that catalog's own static plan in most settings where the requirement can be met, by about a fifth with three HNSW indexes and about a third with a single index (Table 16). With a single index, this saving needs no additional memory, and labeling the construction sample costs far less, since only three configurations are run. Where the single index's static plan is already its cheapest setting (SigLIP2 and OpenCLIP i⟶i), and for OpenCLIP t⟶i, the synopsis saves nothing and costs 3–8% more, mostly its lookup; where the static plan is the middle setting for Jina-v4 and Qwen3-VL i⟶i, it still saves 19% and 32%.

Summary. The saving does not depend on a large catalog or a weak static baseline. Against the best static choice among all indexes we built, the synopsis still costs a median of 0.70×; a single HNSW index searched at three settings, a quarter of the full catalog's memory, lets the synopsis save within that index alone. The single index is not a uniform substitute for the full catalog, however: with one grid of `efSearch` values per model it cannot meet the requirement in three settings, and per setting it is much cheaper or much costlier than the full catalog.

5.5 RQ5: What does the synopsis cost, and how sensitive is it to its parameters?

Lookup cost. In the batched regime used throughout, the lookup takes 21–32 μs per query (Table 17), at most a fifth of the cheapest configuration's cost, and it is included in every cost of Table 13. One query at a time, it is dominated by the exact scan of the K landmarks: it costs less than `hnsw_low` for OpenCLIP and SigLIP2, but several times as much for the 2,048-dimensional Jina-v4 and Qwen3-VL (italic). An HNSW index over the landmarks makes the single-query lookup 3.1× (SigLIP2 t⟶i) to 11.3× (Jina-v4 i⟶i) faster (timed on a shared node). The budget rule then chooses the same

K	Skill	Thin	Size (MiB)
1	0.000	–	<0.1
500	0.280	0.01%	4.5–9.4
1,250	0.335	0.02%	11.3–23.5
2,500	0.374	0.04%	22.5–47.0
5,000	0.416	0.14%	45.1–93.9
10,000	**0.451**	**0.51%**	**90.2–187.8**
20,000	0.469	2.17%	180.4–375.7
25,000	0.468	3.61%	225.4–469.6
40,000	0.440	10.46%	360.7–751.3

Table 18: Sensitivity to the number of landmarks K over the 16 settings ($K = 1$ is a single cell, the global average). Skill: mean over settings of the mean over all nine configurations. Bold: the default. Thin: held-out queries in cells with fewer than $q_{\min}$ sample queries. Terms: Table 8.

configuration for at least 98.5% of queries, and its relative cost and violation rate change by at most 0.003 and 0.05 percentage points. Only for Qwen3-VL does the lookup still cost more than `hnsw_low` (1.1–1.5×). Table 19 gives the timings in every setting.

Size and construction. The synopsis occupies 90–188 MiB, mostly the landmarks themselves, well under 1% of the HNSW indexes of a single tier. Building it requires a few minutes of GPU time for clustering and an hour or less for exact search on Q, shared by all configurations. Running every configuration on Q dominates the offline cost, most of all for SigLIP2 and OpenCLIP, whose IVF-family configurations probe thousands of partitions; with a single index (Section 5.4.1), it is several times cheaper. This cost is paid once per model and catalog.

Number of landmarks. Skill rises with K up to about 20,000 landmarks and then falls, as more held-out queries land in cells with fewer than $q_{\min}$ sample queries (Table 18). Lookup cost and synopsis size grow linearly with K; our default of 10,000 is close to the best skill at half the cost of 20,000.

Construction sample. The synopsis needs a large construction sample at this K: built from 10% of Q, most held-out queries fall into sparse cells and skill drops by more than half, and with 1% the synopsis is barely better than the global average. Roughly 70 sample queries per cell are needed. The minimum support matters less: $q_{\min} = 1$ leaves skill unchanged, while $q_{\min} = 50$ lowers it.

Answer to RQ5. The synopsis is small, and its batched lookup costs at most a fifth of the cheapest configuration; its one-time construction is dominated by running the catalog on the construction sample, which needs several hundred thousand queries per direction at $K = 10{,}000$. Accuracy grows with K as long as cells keep enough sample queries. A single-query lookup by exact landmark scan is too slow for 2,048-dimensional models. An HNSW index over the landmarks makes it 3–11× faster.

6 A REALISTIC DEPLOYMENT

Scenario. Our nine separately built indexes measure how large the opportunity is. A practical deployment needs much less. Consider a vector database that serves a large multimodal collection at high volume and in batches, as retrieval-augmented generation and semantic query pipelines do. It keeps one HNSW index and exposes several `efSearch` settings, which vector databases already support. The construction sample is drawn from the corpus itself, so no query log is needed, and labeling runs only those settings. The synopsis is stored with the index, where a relational optimizer keeps its statistics. At query time, the operator reads the query's cell and dispatches the query to the setting that the cell's precomputed choice names. The interface of the operator does not change.

Evidence. Our single-index results cover this scenario (Section 5.4.1). At no extra memory, the synopsis costs a median of 0.68× the index's own static plan in the 13 settings where the index can meet the requirement, and labeling the sample costs 84–705 CPU-hours rather than 344–5,036. The construction sample needs about 70 queries per cell (Section 5.5), independent of the corpus size, so for a corpus of hundreds of millions of items it is a small fraction of the collection. Labeling is a one-time cost. From the per-configuration costs of Table 5 and the savings of Table 13, we estimate that labeling the full catalog is repaid after a median of about 40 million queries per setting, from about 6 million for cross-modal settings with an expensive static plan to more than 100 million in five settings (SigLIP2 i→i and t→t, OpenCLIP i→i, Qwen3-VL i→i and i→t; at most 394 million). For SigLIP2 and OpenCLIP, labeling is dominated by IVF-family configurations that are rarely chosen. The single index, in the 10 settings where it saves, is repaid after a median of about 2 million queries. This estimate counts only the CPU time of running the configurations on the sample. Exact search on the sample adds about an hour of GPU time per model.

Other targets and costs. Each cell keeps a histogram, so one synopsis serves any fidelity target without being rebuilt. A new target or reliability level only requires retuning λ on the sample's stored labels. Cost enters only through $c(A_j)$ in the budget rule. Latency could be replaced by energy, or by a cost that charges memory or I/O, as for disk-resident indexes [7, 22], without changing the synopsis.

Keeping the synopsis current. A new configuration adds one column, built by running it on the sample, whose exact neighbors are already known. Inserts and deletes in the corpus change both the exact neighbors and the indexes. Small changes should leave cell statistics roughly valid, much as histograms tolerate small updates. Large changes call for relabeling, which can be confined to the cells whose items changed. A shift in the query workload moves queries between cells, but it does not change what a cell's statistics mean, as long as new queries resemble the sample queries in the same cell.

What remains untested. Two parts of this scenario are not covered by our experiments. Real user queries, such as short typed text, may differ from items sampled from the corpus. Our cross-modal results suggest caution, since estimates are less accurate for queries far from all landmarks (Section 5.2). And our costs are estimated from per-configuration latencies measured with full batches, not measured end to end for a selector that splits each batch across indexes or settings. We have also not evaluated the maintenance strategies above.

		Exact scan (μs)		HNSW over landmarks (μs)			Same	Same configuration chosen			
Model (HNSW parameters)	Dir.	exclusive	shared	mean	p99	Speed-up	cell	Mean	Coverage	Budget (static)	Budget ($\leq 1-\rho$)
SigLIP2 (M=16, ef=256)	i⟶i	1,626	1,604	**495**	620	3.2×	99.96%	100.00%	99.99%	99.99%	**99.99%**
	i⟶t	1,629	1,572	**442**	500	3.6×	97.49%	98.84%	98.61%	98.54%	**98.54%**
	t⟶i	1,628	1,597	**510**	599	3.1×	99.35%	99.80%	99.66%	99.76%	**99.97%**
	t⟶t	1,642	1,588	**494**	597	3.2×	99.88%	99.98%	99.95%	99.98%	**99.98%**
OpenCLIP (M=16, ef=128)	i⟶i	885	936	**225**	291	4.2×	99.86%	99.95%	99.92%	99.97%	**99.96%**
	i⟶t	791	958	**236**	279	4.1×	98.92%	99.31%	99.53%	99.58%	**99.40%**
	t⟶i	779	953	**240**	287	4.0×	98.11%	99.49%	99.03%	99.03%	**99.85%**
	t⟶t	792	947	**234**	281	4.1×	99.89%	99.95%	99.94%	99.95%	**99.97%**
Jina-v4 (M=32, ef=64)	i⟶i	2,845	2,979	**263**	381	11.3×	99.94%	99.96%	99.95%	99.96%	**99.96%**
	i⟶t	2,847	2,981	**303**	422	9.8×	99.85%	99.88%	99.91%	99.94%	**99.91%**
	t⟶i	2,848	2,984	**275**	390	10.8×	99.52%	99.67%	99.80%	99.86%	**99.76%**
	t⟶t	2,846	2,983	**306**	451	9.7×	99.68%	99.75%	99.79%	99.79%	**99.80%**
Qwen3-VL (M=32, ef=128)	i⟶i	2,827	2,779	**492**	656	5.7×	99.98%	99.99%	99.99%	99.99%	**99.99%**
	i⟶t	2,834	2,828	**397**	530	7.1×	99.79%	99.85%	99.86%	99.87%	**99.88%**
	t⟶i	2,830	2,784	**527**	679	5.3×	99.88%	99.92%	99.97%	99.98%	**99.96%**
	t⟶t	2,830	2,806	**465**	694	6.0×	99.89%	99.92%	99.93%	99.94%	**99.93%**

Table 19: Single-query lookup (Step 1, one thread) with an HNSW index over the $K = 10{,}000$ landmarks of each modality, one setting per model for all four directions: the fastest whose choice under the coverage rule agrees with the exact scan's on at least 99.9% of a 5,000-query held-out sample in at least two directions. Exact scan, exclusive: on an otherwise idle node, as in Table 17; shared: on a busy node, like the HNSW search. Speed-up: the shared exact scan divided by the HNSW mean. Same cell: held-out queries located in the same cell as by the exact scan. Same configuration chosen: held-out queries for which each selection rule picks the same configuration with either lookup, at $\tau = 0.7$. Bold: the HNSW lookup time and the agreement for the budget rule at the requirement, our method.

7 RELATED WORK

Approximate nearest-neighbor indexes and their tuning. Graph-based indexes such as HNSW [30] and DiskANN [22], and partition- and quantization-based ones such as IVF with product quantization [15] and its optimized variant [9], SPANN [7], and ScaNN [13], cover a wide range of trade-offs between cost and fidelity. Benchmarks and surveys compare them by their average recall over a workload [2, 34, 44], and tuners such as VDTuner [48] search index types and parameters for a workload-level target of throughput and recall. All of these choose one configuration for a workload; they produce a better static plan. Our synopsis takes a catalog as given and chooses among its configurations per query. The two are complementary: a better-tuned catalog gives the synopsis better options, as the union catalog of Section 5.4.1 shows.

Adaptive search within an index. Learned adaptive early termination [25] predicts, per query, when a search has found its neighbors, and DARTH [4] terminates a search once a declared target recall is predicted to be met. Ada-ef [50] sets HNSW's `efSearch` per query from offline statistics of the distance distribution, and the adaptive partition scanning of Quake [31] sets the number of partitions an IVF-style index scans per query from a running estimate of recall. All of these adapt how far a search goes inside an index that has already been chosen, and all need that index to be able to reach the target. Closest to our setting, MINT [52] plans multi-vector queries across several indexes, choosing which indexes to use and how many items to take from each under a recall threshold. Its recall estimates come from models fitted on a sample of the data rather than from the query's own surroundings, and it does not hold a workload to a miss rate. Plan selection makes the earlier decision, which index and which settings to use, before any search runs, and it can move a query to a different index family when the default one cannot reach the target. The approaches compose: an early-termination method can run inside whichever configuration the synopsis selects.

Query hardness. Local intrinsic dimensionality [3] and Steiner-hardness [45] characterize how hard a query is for nearest-neighbor search, the latter for graph indexes specifically, and work on cross-modal search observes that queries of one modality lie outside the distribution of the other [6, 27]. These measures explain why some queries are hard, but they are not calibrated estimates of whether a particular configuration will meet a particular target, which is what plan selection needs. The synopsis measures that quantity directly, per region and per configuration; in our experiments, local density alone predicts fidelity much less well (Section 5.2).

Cardinality estimation and synopses. Cost-based optimizers choose among access paths using estimates of each plan's output size [39], obtained from synopses such as histograms built offline and consulted at plan time [14], and more recently from learned models [21]; the quality of these estimates largely determines the quality of the chosen plans [23]. The fidelity synopsis applies this design to vector search, with cells of the embedding space as buckets and fidelity as the statistic. Unlike selectivity, which is a property of the data and the predicate, fidelity also depends on the physical plan, so the synopsis is keyed by configuration as well as by region.

Accuracy-aware query processing. A broad line of work trades accuracy for cost under an explicit target: approximate query processing with error bounds [1], including over multimodal data [16];

cheap filters and model cascades for machine-learning inference [18, 29]; cascades and routing among language models [5]; and optimizers for semantic operators that choose models and plans for cost and quality [28, 35, 40]. Our work brings this view to the vector-search operator itself, with a per-query choice among physical configurations and a workload-level requirement on how often the fidelity target may be missed.

Vector search in database systems. Vector databases and extensions such as Milvus [43], AnalyticDB-V [46], and pgvector [41] expose index types and search parameters, typically as settings for a collection or session [34]. Work on hybrid queries, such as VBase [51] and ACORN [36], chooses how to combine vector search with relational predicates, based on the predicates' selectivity. To our knowledge, no system estimates, before execution, how each candidate configuration will perform on a single query from where that query lies, and chooses among index families under a requirement on the whole workload.

8 LIMITATIONS AND CONCLUSION

Limitations. Our costs are estimated from per-configuration latencies measured with full batches, not measured end to end for a deployed selector that splits each batch across indexes. Half of the variation in fidelity lies within cells, so the synopsis captures only part of the per-query optimum's saving, and less when held to the static plan's realized violation rate than to the requirement. The comparison at the requirement lets the synopsis spend an allowance the static plan leaves unused; we report the stricter comparison as well. Building the synopsis requires labeling a large construction sample with every configuration, a one-time cost that is substantial for a large catalog and much smaller for a single index. With an exact scan of the landmarks, the single-query lookup is too slow for high-dimensional models. An HNSW index over the landmarks removes most of this cost. Finally, the size of the opportunity depends on the catalog: part of it, for some models, comes from how the catalog's HNSW tiers were built, although the synopsis's advantage persists against the best static choice among all the indexes we built.

Scope of the evaluation. Our results come from one corpus, LAION image–caption pairs, with plan selection at one result depth, $k = 100$ (estimation accuracy also at $k = 10$). They cover three index families in one library, FAISS, with all indexes in memory on CPUs. The four models and four directions give 16 settings that differ widely in dimension, modality gap, and cost order, and the conclusions hold across all of them. Other corpora, smaller result depths, disk-resident indexes, and GPU search may change the size of the opportunity. They do not change how the synopsis is built or used. The construction and held-out queries are images and captions drawn from the same distribution as the corpus. Real workloads, such as short typed text queries, may differ. Cross-modal search is a partial test of such a shift, since its queries lie away from the items they search. The estimates are less accurate there, and for queries far from all landmarks (Section 5.2). A deployment should therefore track the share of queries that fall far from their nearest landmark, and extend the construction sample when that share grows. Our costs also assume that every index receives full batches, which per-query selection may not provide under light load.

Future work. Three directions follow directly. Reducing the landmarks' dimension before indexing them would make single-query lookups cheaper still for high-dimensional models. Search-parameter grids chosen per query direction, rather than per model, would remove the coarse steps that limited the single-index catalog. And reducing the labeling cost, by sampling cells adaptively or by transferring a synopsis between catalogs that share an index, would make the synopsis cheaper to build and to keep current as the corpus changes.

Conclusion. A static ANN deployment must be sized for its hardest queries and therefore overpays for the rest. We showed that a query's location in the embedding space predicts how each configuration will perform on it, that this prediction must be kept per configuration, and that a compact synopsis of it, consulted with a single nearest-landmark lookup, lets an optimizer choose configurations per query while meeting the requirement a static deployment is built to meet, at a median estimated cost of 0.72× the static plan. The saving persists against the best static choice among the same indexes, and within a single index wherever that index can meet the requirement. A fidelity synopsis is to vector search what a histogram is to relational selectivity estimation: a statistic built once, offline, that an optimizer consults before executing any plan.

REFERENCES

[1] Sameer Agarwal, Barzan Mozafari, Aurojit Panda, Henry Milner, Samuel Madden, and Ion Stoica. BlinkDB: Queries with bounded errors and bounded response times on very large data. In *Eighth Eurosys Conference 2013, EuroSys '13*, pages 29–42. ACM, 2013.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.*, 87:101374, 2020.

[3] Martin Aumüller and Matteo Ceccarello. The role of local dimensionality measures in benchmarking nearest neighbor search. *Inf. Syst.*, 101:101807, 2021.

[4] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3(4):242:1–242:26, 2025.

[5] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *CoRR*, abs/2305.05176, 2023.

[6] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. RoarGraph: A projected bipartite graph for efficient cross-modal approximate nearest neighbor search. *Proc. VLDB Endow.*, 17(11):2735–2749, 2024.

[7] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: highly-efficient billion-scale approximate nearest neighborhood search. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 5199–5212, 2021.

[8] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pages 2818–2829. IEEE, 2023.

[9] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization. *IEEE Trans. Pattern Anal. Mach. Intell.*, 36(4):744–755, 2014.

[10] W Brier Glenn et al. Verification of forecasts expressed in terms of probability. *Monthly weather review*, 78(1):1–3, 1950.

[11] Michael Günther, Saba Sturua, Mohammad Kalim Akram, Isabelle Mohr, Andrei Ungureanu, Bo Wang, Sedigheh Eslami, Scott Martens, Maximilian Werk, Nan Wang, and Han Xiao. jina-embeddings-v4: Universal embeddings for multimodal multilingual retrieval. In David Ifeoluwa Adelani, Catherine Arnett, Duygu Ataman, Tyler A. Chang, Hila Gonen, Rahul Raja, Fabian Schmidt, David Stap, and Jiayi Wang, editors, *Proceedings of the 5th Workshop on Multilingual Representation Learning (MRL 2025)*, pages 531–550, Suzhuo, China, November 2025. Association for Computational Linguistics.

[12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330. PMLR, 2017.

[13] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896. PMLR, 2020.

[14] Yannis E. Ioannidis. The history of histograms (abridged). In *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*, pages 19–30, 2003.

[15] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011.

[16] Saehan Jo and Immanuel Trummer. Thalamusdb: Approximate query processing on multi-modal data. *Proc. ACM Manag. Data*, 2(3):186, 2024.

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *CoRR*, abs/1702.08734, 2017.

[18] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. NoScope: Optimizing deep CNN-based queries over video streams at scale. *Proc. VLDB Endow.*, 10(11):1586–1597, 2017.

[19] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 6769–6781. Association for Computational Linguistics, 2020.

[20] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over BERT. In Jimmy X. Huang, Yi Chang, Xueqi Cheng, Jaap Kamps, Vanessa Murdock, Ji-Rong Wen, and Yiqun Liu, editors, *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 39–48. ACM, 2020.

[21] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019*, 2019.

[22] Ravishankar Krishnaswamy, Magdalen Dobson Manohar, and Harsha Vardhan Simhadri. The diskann library: Graph-based indices for fast, fresh and filtered vector search. *IEEE Data Eng. Bull.*, 48(3):20–42, 2024.

[23] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proc. VLDB Endow.*, 9(3):204–215, 2015.

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[25] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2539–2554, 2020.

[26] Mingxin Li, Yanzhao Zhang, Dingkun Long, Keqin Chen, Sibo Song, Shuai Bai, Zhibo Yang, Pengjun Xie, An Yang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Qwen3-vl-embedding and qwen3-vl-reranker: A unified framework for state-of-the-art multimodal retrieval and ranking. *CoRR*, abs/2601.04720, 2026.

[27] Weixin Liang, Yuhui Zhang, Yongchan Kwon, Serena Yeung, and James Zou. Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[28] Chunwei Liu, Matthew Russo, Michael J. Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael J. Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. A declarative system for optimizing AI workloads. *CoRR*, abs/2405.14696, 2024.

[29] Yao Lu, Aakanksha Chowdhery, Srikanth Kandula, and Surajit Chaudhuri. Accelerating machine learning inference with probabilistic predicates. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD 2018*, pages 1493–1508, 2018.

[30] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(4):824–836, 2020.

[31] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. Quake: Adaptive indexing for vector search. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025*, pages 153–169. USENIX Association, 2025.

[32] Allan H. Murphy. Skill scores based on the mean square error and their relationships to the correlation coefficient. *Monthly Weather Review*, 116(12):2417–2424, 1988.

[33] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 29, 2015.

[34] James Jie Pan, Jianguo Wang, and Guoliang Li. Survey of vector database management systems. *VLDB J.*, 33(5):1591–1615, 2024.

[35] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. LOTUS: enabling semantic queries with llms over tables of unstructured and structured data. *CoRR*, abs/2407.11418, 2024.

[36] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data*, 2(3):120, 2024.

[37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 2021.

[38] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. LAION-400M: open dataset of clip-filtered 400 million image-text pairs. *CoRR*, abs/2111.02114, 2021.

[39] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979.

[40] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. Docetl: Agentic query rewriting and evaluation for complex document processing. *Proc. VLDB Endow.*, 18(9):3035–3048, 2025.

[41] The pgvector Contributors. pgvector: Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector.

[42] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia,

Basil Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.

[43] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, pages 2614–2627, 2021.

[44] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.*, 14(11):1964–1978, 2021.

[45] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. Steiner-hardness: A query hardness measure for graph-based ANN indexes. *Proc. VLDB Endow.*, 17(13), 2024.

[46] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proc. VLDB Endow.*, 13(12):3152–3165, 2020.

[47] Mengyao Xu, Gabriel de Souza Pereira Moreira, Ronay Ak, Radek Osmulski, Yauhen Babakhin, Zhiding Yu, Benedikt Schifferer, and Even Oldridge. Llama nemoretriever colembed: Top-performing text-image retrieval model. *CoRR*, abs/2507.05513, 2025.

[48] Tiannuo Yang, Wen Hu, Wangqi Peng, Yusen Li, Jianguo Li, Gang Wang, and Xiaoguang Liu. VDTuner: Automated performance tuning for vector data management systems. In *40th IEEE International Conference on Data Engineering, ICDE 2024*, 2024.

[49] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 11941–11952. IEEE, 2023.

[50] Chao Zhang and Renée J. Miller. Distribution-aware exploration for adaptive HNSW search. *CoRR*, abs/2512.06636, 2025. To appear in SIGMOD 2026.

[51] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. VBase: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023*, pages 377–395, 2023.

[52] Jiongli Zhu, Yue Wang, Bailu Ding, Philip A. Bernstein, Vivek Narasayya, and Surajit Chaudhuri. MINT: Multi-vector search index tuning. *CoRR*, abs/2504.20018, 2025.